

top

КОМПЬЮТЕРНАЯ
АКАДЕМИЯ

ОБЪЕКТНО -ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ НА C++

Урок 5

Шаблоны классов,
класс string

Содержание

1. Статический полиморфизм	
и шаблоны как частный случай.....	5
Идея полиморфизма	5
Статический и динамический полиморфизм	7
Шаблоны функций	8
2. Шаблоны классов	18
Шаблоны классов.....	18
Требования шаблона к используемым параметрам-типам. Стандартная библиотека шаблонов STL	36
Специализация шаблона функции.....	45
Полная специализация шаблона класса.....	50
Частичная специализация шаблона класса.....	58
Примеры создания шаблонов классов	66
3. Шаблоны с переменным числом аргументов.....	85
4. Использование std::initializer_list.....	92

5. Класс <code>string</code>	99
Представление символов, таблицы кодировок	99
Символьные типы данных и строки C++	103
Цели и задачи класса <code>string</code>	105
Анализ устройства класса <code>string</code>	106
Объявление, инициализация, доступ к элементам	106
Ввод и вывод	109
Операторы	110
Методы класса <code>string</code>	112
Методы доступа к элементам	112
Методы-итераторы	114
Методы поиска	115
Методы редактирования	119
Методы работы со вместимостью	128
Сервисные функции (не члены класса)	132
Примеры использования класса <code>string</code>	136
Программа поиска в тексте	136
Программа создания частотного словаря текста	141
6. Резюме	147
Раздел 1. Статический полиморфизм и шаблоны как частный случай	147
Раздел 2. Шаблоны классов	148
Раздел 3. Шаблоны с переменным числом аргументов	152
Раздел 4. Использование <code>std::initializer_list</code>	153

Раздел 5. Класс string.....	154
Методы доступа к символам строки:	156
Методы — итераторы	156
Методы поиска в строке:	156
Методы редактирования строки	157
Методы работы со вместимостью	157
Сервисные функции (не члены класса)	158
7. Домашнее задание.....	159
1. Сжатие текста	159
2. Удаление комментариев	159
3. Поиск и выделение	159
4. Вычисление сумм.....	160
5. Римские числа	160
6. Полное имя (путь) файла	160
8. Терминология	162

Материалы урока прикреплены к данному PDF-файлу. Для доступа к ним, урок необходимо открыть в программе Adobe Acrobat Reader.

1. Статический полиморфизм и шаблоны как частный случай

Полиморфизм в языках программирования и теории типов — способность функции обрабатывать данные разных типов.

Википедия

Идея полиморфизма

Полиморфизм. Дословный перевод с первоначального (древнегреческого) значения — много форм.

В наиболее общем виде полиморфизм обозначает:

- либо выполнение одной и той же функции различными формами (например, часы — песочные, механические, электронные, — выглядят очень по-разному, но все делают одно и то же — измеряют время);
- либо объединение нескольких форм в одной сущности (мифический Минотавр — человеко-бык).

В программировании полиморфизм понимается в хорошем, первом смысле: полиморфным является код, который во время исполнения программы может обрабатывать значения различных типов.

Иллюстрацией такого полиморфизма является вышеприведенный пример с часами: различные часовые механизмы, все, — определяют и показывают время, хотя получают и обрабатывают сигналы разного типа (поток песка, колебания маятника, электромагнитные колебания).

Полиморфизм в программе может быть выражен в разной степени. Проследим последовательное развитие

этого качества — от полного отсутствия до максимально возможной величины.

Представьте, что нам потребовалось разработать функцию сортировки массива целых чисел. Разработали, отладили... Прототип функции будет выглядеть следующим образом:

```
void sort(int* intArray, int size);
```

А потом нам потребовалась еще и функция сортировки массива чисел с плавающей запятой.

Без поддержки полиморфизма в языке программирования нам пришлось бы придумывать еще одно название функции — функция-то другая, поскольку типы параметров не совпадают:

```
void sortD(double* dArray, int size);
```

А потом еще... потребовалась функция сортировки строк:

```
void sortS(string* sArray, int size);
```

Как-то неудобно получается, — несколько функций с одинаковым, в общем-то назначением, с практически одинаковым кодом, называются по-разному, нужно помнить, где какую применять...

Понятно, что в данном конкретном случае с этими трудностями справиться легко, но представьте себе вполне обычный проект с десятками разработчиков, тысячами функций и множеством ситуаций, где функции различаются только чуть-чуть — подобно рассмотренному выше примеру.

Вот здесь нам на помощь и приходит полиморфизм, — в начальной степени, в виде простой перегрузки функций.

```
void sort(int* array, int size);  
void sort(double* array, int size);  
void sort(string* array, int size);
```

С использованием перегрузки функций программа будет выглядеть лучше, — сокращается количество понятий и имен. Теперь у нас есть только одно имя функции, одно назначение функции, один вариант вызова, один интерфейс и несколько вариантов реализации. С таким подходом становится проще писать код и модифицировать его при необходимости.

Статический и динамический полиморфизм

Нужно понимать, что полиморфизм — это не уникальная особенность C++ и не только один из основных принципов ООП. Полиморфизм — это важное понятие теории и практики разработки программ. Полиморфизм поддерживается в разных языках и технологиях программирования.

В природе программирования имеется два типа полиморфизма, существенно различающихся между собой:

- полиморфизм времени выполнения (или динамический полиморфизм);
- полиморфизм времени компиляции (или статический полиморфизм).

Динамический полиморфизм реализуется во время работы программы. Он представляет собой использование так называемых виртуальных функций, работа с

которыми осуществляется через указатели. С виртуальными функциями мы познакомимся позже.

Статический полиморфизм реализуется во время компиляции программы.

C++ предоставляет три вида статического полиморфизма:

- перегрузка функции (*function overloading*) ;
- перегрузка оператора (*c++ operator overloading*);
- шаблоны (*templates*).

Существуют шаблоны функций (*function templates*) и шаблоны классов (*class templates*).

Шаблон функции представляет собой запись алгоритма, который может выполняться с разными типами данных. Тип данных является специальным параметром-типом шаблона. Подставляя различные типы данных в этот шаблон, мы получаем целое семейство разных функций с одинаковым алгоритмом и разными типами данных.

Шаблон класса представляет собой обобщенное описание класса, которое, (как и шаблон функции) имеет смысл для разных типов данных, используемых для представления полей и функций класса.

Шаблоны функций

Итак, у нас есть три функции-близнеца, которые одинаковым образом сортируют массивы разных типов.

```
void sort(int* array, int size);  
void sort(double* array, int size);  
void sort(string* array, int size);
```

Если бы коды этих функций различались между собой, то на этом и пришлось бы остановиться. Но они — идентичны, за исключением того, что переменные различаются типом данных. На этот случай в C++ и предусмотрены шаблоны функций. Мы напишем только одну функцию (один шаблон функции), в которой предусмотрим возможность использования разных типов данных путем задания специальных параметров — параметров-типов шаблона.

Безусловно, один текст функции — это гораздо лучше, нежели три, особенно в смысле сопровождения программы.

Дублирование кода — это вообще смертный грех программиста. Программы подвержены изменениям, а при изменении одного участка кода легко забыть о том, что подобное же изменение нужно сделать и в другом месте. В результате — ошибка, на поиски и исправление которой понадобится время и ресурсы, которых всегда не хватает...

Шаблон функции начинается с ключевого слова `template`, за которым в угловых скобках следует список параметров-типов. Параметры-типы указываются с помощью ключевых слов `typename` или `class` (можно использовать и то, и другое, разницы между ними нет).

Затем следует определение функции, в котором для переменных с изменяемыми типами данных вместо указания конкретных типов используются имена параметров-типов.

```
template < template-parameter-list > function-declaration
```

Например, шаблон функции вычисления квадрата числа:

```
template<class T>

T square(T number)
{
    T result = number * number;
    return result;
}
```

Здесь:

- `class T` — определение параметра-типа,
- `T` — изменяемый тип данных. Функция может возводить в квадрат и целые, и плавающие числа.

Итак, продолжим процесс развития полиморфизма, — напомним шаблон функции `sort`, упомянутой ранее.

```
template <class T>

void sort(T array[], size_t size)
{
    for (int k = size - 1; k > 0; k--)
    {
        for (int j = 0; j < k; j++)
        {
            if (array[j] > array[j + 1])
            {
                swap(array[j], array[j + 1]);
            }
        }
    }
}
```

Разработка шаблона функции состоит из следующих шагов:

- создается функция для конкретного типа (в нашем примере — пузырьковая сортировка для типа `int`);
- перед заголовком функции добавляется список параметров шаблона — ключевое слово `template` и список параметров-типов шаблона в угловых скобках (в нашем случае — `template <class T>`);
- в реализации функции имя типа (`int`) заменяется на имя параметра-типа (`T`).

Итак, мы достигли высшего уровня полиморфизма, — у нас имеется только один код, решающий одну и ту же задачу для нескольких, разных наборов данных.

Аналогично, разрабатываем шаблон функции вывода массива на экран.

```
template<class T>
void display(T array[], size_t size)
{
    for (int i = 0; i < size; i++)
    {
        cout << array[i] << " ";
    }
    cout << endl;
}
```

Теперь можно использовать разработанные шаблоны для сортировки и вывода на экран массивов разных типов.

(Используемый в приведенном примере тип `size_t` — это беззнаковый целый тип, который обычно применяется

для счетчиков циклов, индексации массивов, хранения размеров объектов).

Вызов (применение) шаблонной функции выполняется так же, как и вызов обычной функции: указывается имя функции и список значений параметров в скобках.

Но при этом, в общем случае, нужно указать значения параметров-типов шаблона. Эти значения записываются в угловых скобках между именем функции и списком параметров, например, `sort<int>(intArray, size)` — вызов функции `sort` для обработки массива целых чисел.

Приведем код, использующий разработанные шаблоны (полный код программы — в папке *Lesson05\les05_01*):

```
int main()
{
    cout << "Sort Template" << endl << endl;

    int intArray[]{ 1, 3, 7, -4, -2, 4 };
    int size = sizeof(intArray) / sizeof(int);
    cout << "Original int Array : ";
    display<int>(intArray, size);
    sort<int>(intArray, size);
    cout << "Sorted   int Array : ";
    display<int>(intArray, size);

    char chrArray[]{ 'o', 't', 't', 'f', 'f', 's', 's',
                     'e', 'n' };
    size = sizeof(chrArray) / sizeof(char);
    cout << "Original chr Array : ";
    display<char>(chrArray, size);
    sort<char>(chrArray, size);
    cout << "Sorted   chr Array : ";
    display<char>(chrArray, size);

    string strArray[]{ "one", "two", "three", "four", "five" };
```

```

size = sizeof(strArray) / sizeof(string);
cout << "Original str Array : ";
display<string>(strArray, size);
sort<string>(strArray, size);
cout << "Sorted str Array : ";
display<string>(strArray, size);

_getch();
return 0;
}

```

Помимо встроенных типов данных `int` и `char` в приведенном примере используется тип `string`, с которым вы вскоре подробно познакомитесь. `string` — это класс из стандартной библиотеки C++, предназначенный для работы с текстовыми данными. Назначение `string` такое же, как и у массива символов (`char[], char*`), но тип `string` имеет более развитые средства для работы с текстом.

Результат работы программы приведен на рисунке 1.

Sort Template

```

Original int Array : 1 3 7 -4 -2 4
Sorted int Array : -4 -2 1 3 4 7
Original chr Array : o t t f f s s e n
Sorted chr Array : e f f n o s s t t
Original str Array : one two three four five
Sorted str Array : five four one three two

```

Рисунок 1. Использование шаблона функции `sort` для сортировки массивов разных типов

Указывать значение параметра-типа, как это сделано в следующих строках, — необязательно.

```
display<int>(intArray, size);  
sort<int>(intArray, size);
```

Можно просто записать:

```
display(intArray, size);  
sort(intArray, size);
```

Компилятор по типу параметра (`intArray` — `int`) «поймет», для какого типа нужно сгенерировать функцию из шаблона. Но в некоторых случаях может возникнуть неоднозначность, и тогда указание типа-параметра будет необходимо.

Важной особенностью шаблона функции является то, что до тех пор, пока нет вызова функции, шаблон не компилируется.

Шаблон компилируется для каждого параметра-типа, для которого он вызывается (в приведенном выше примере компилируется 3 версии функции `sort` — для `int`, `string` и `char`).

Если шаблон функции используется (вызывается) в нескольких файлах проекта, то тело шаблона функции необходимо размещать в файле-заголовке (в `*.h` файле, а не в `*.cpp` файле).

Если мы поступим с шаблоном функции, как с обычной функцией, то есть поместим тело шаблона функции в файле `*.cpp`, а прототип шаблона функции — в файле вызывающей функции (код примера — в папке

Lesson05\les05_01a), то компиляция вызова шаблона пройдет нормально, а вот сборка завершится сообщениями об ошибках, как показано на рисунке 2:

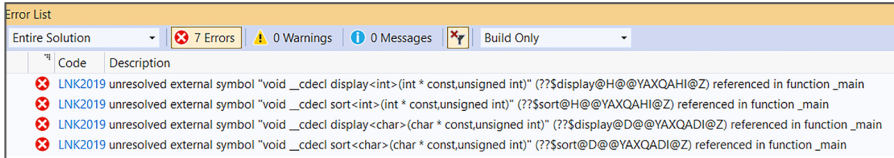


Рисунок 2. Ошибка сборки программы с размещением тела шаблона функции в файле *.cpp

В самом деле:

- шаблон функции компилируется только при вызове,
- компилятор при компиляции вызова шаблонной функции не видит тело шаблона функции, размещенное в другом файле, и не компилирует его при вызове,
- компилятор не компилирует шаблон без вызова.

Следовательно, шаблон функции в этом случае не компилируется вообще, и сборщик (линкер) закономерно сообщает, что вызываемой функции нет.

Приведем еще пример шаблона функции.

Шаблон функции `getValue` для ввода данных разных типов с клавиатуры:

```
template <class T>
void getValue(string prompt, T& value)
{
    cout << prompt;
    cin >> value;
```

```

while (cin.fail())
{
    cin.clear();
    cin.ignore(32767, '\n');
    cout << "Error... try again" << endl;

    cout << prompt;
    cin >> value;
}

string endLine;
getline(cin, endLine);
}

```

В приведенном примере используются библиотечные функции обработки потока ввода:

- функция `cin.fail()` возвращает `true` в случае ошибки ввода;
- функция `cin.clear()` восстанавливает поток ввода из «ошибочного» в нормальное «рабочее» состояние;
- функция `cin.ignore()` очищает буфер ввода от оставшихся не введенных символов.

Все это вместе, организованное в цикл, вынуждает пользователя в случае ошибки ввода повторять ввод в до тех пор, пока ошибки не будет. Выход из цикла (или обход его) — происходит только при правильном вводе значения.

Данная функция ориентирована на ввод одиночного значения и нажатие клавиши «Enter» («Ввод») после ввода. При считывании значения (`cin >> value;`) символ «Enter» ('\n' — конец строки) остается в потоке ввода. Для

извлечения этого символа из потока и нужны последние две строки шаблона.

```
string endLine;  
getline(cin, endLine);
```

Функция `getline(cin, endLine)` считывает в переменную `endLine` (типа `string`) все, что есть до ближайшего символа `'\n'`, включая и сам этот символ. Таким образом, поток ввода будет очищен и готов к вводу следующих значений.

С помощью этого шаблона удобно организовывать ввод данных разных типов, например

```
int number;  
getValue("Enter number: ", number);  
  
string name;  
getValue("Enter name: ", name);
```

Использование полиморфизма во всех его разновидностях делает программы более ясными и лучше сопровождаемыми. Применение шаблонов C++, в дополнение к этому, приводит к более коротким и легче изменяемым программам.

Шаблоны несколько трудны для изучения. Однако, преимущества, которые можно извлечь из их использования, перевешивают все трудности. Следуют также помнить, что шаблоны широко используются в стандартных библиотеках C++, а написание современных программ немислимо без активного применения этих библиотек.

2. Шаблоны классов

Обобщённое программирование (англ. generic programming) — парадигма программирования, заключающаяся в таком описании данных и алгоритмов, которое можно применять к различным типам данных, не меняя само это описание.

Википедия

Шаблоны классов

Обобщенное программирование еще называют универсальным программированием. В языке C++ обобщенное программирование основывается на шаблонах функций и шаблонах классов.

Шаблоны классов позволяют создавать описание абстрактного класса, которое при использовании шаблона разворачивается компилятором в конкретные классы, реализующие заданные функции для различных типов данных.

Объявление шаблона класса подобно объявлению шаблона функции

```
template < template-parameter-list > class-declaration
```

Например

```
template<class T>
class Array
{
    T* array;
    size_t size;
    . . .
};
```

Шаблон класса начинается с ключевого слова `template`, за которым в угловых скобках следует список параметров шаблона класса. Параметрами шаблонов класса могут быть параметры-типы, параметры обычных типов, параметры-шаблоны.

Далее, в теле класса, — все как обычно, — поля класса, конструкторы, методы. С одним отличием — вместо конкретного указания типа данных в некоторых позициях кода используется имя параметра-типа шаблона (`T`), например,

```
T getItem(size_t index) const
void setItem(size_t index, T value)
```

Рассмотрим простой пример (код программы — в папке *Lesson05\les05_02*):

```
#include <iostream>
#include <conio.h>
using namespace std;

void stopProgram(string message)
{
    cout << message << endl;
    cout << "Press any key to exit program" << endl;
    _getch();
    exit(1);
}

template<class T>
class Array
{
    static const size_t size{ 5 };
    T arr[size];
};
```

```
public:
    Array()
    {
        for (int i = 0; i < size; i++)
        {
            arr[i] = T();
        }
    }

    int getSize() const
    {
        return size;
    }

    T getItem(size_t index) const
    {
        if (index >= 0 && index < size)
        {
            return arr[index];
        }
        else
        {
            stopProgram("Index is out of range!");
        }
    }

    void setItem(size_t index, T value)
    {
        if (index >= 0 && index < size)
        {
            arr[index] = value;
        }
        else
        {
            stopProgram("Index is out of range!");
        }
    }
}
```

```

void display()
{
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void sort()
{
    for (int k = size - 1; k > 0; k--)
    {
        for (int j = 0; j < k; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

};

int main()
{
    cout << "Class Array" << endl << endl;

    Array<int> intArray;
    cout << "int Array initialization:" << endl;
    intArray.display();
    int size = intArray.getSize();
    for (int i = size; i > 0; i--)
    {
        intArray.setItem(size - i, i);
    }
    cout << endl << "int Array after assignment:" << endl;
}

```

```

intArray.display();
intArray.sort();
cout << endl << "int Array after ordering:" << endl;
intArray.display();
cout << endl;

Array<string> strArray;
cout << "str Array initialization:" << endl;
strArray.display();
strArray.setItem(0, "two");
strArray.setItem(1, "seven");
strArray.setItem(2, "zero");
strArray.setItem(3, "four");
strArray.setItem(4, "one");
cout << endl << "str Array after assignment:" << endl;
strArray.display();
strArray.sort();
cout << endl << "str Array after ordering:" << endl;
strArray.display();

cout << endl << "Press any key to exit program" << endl;
_getch();

return 0;
}

```

В этой программе код шаблона класса и его использование, для простоты, располагаются в одном файле. Естественно, код шаблона должен предшествовать функции `main`, в которой вызываются его методы.

Замечания по коду шаблона класса.

- Шаблон класса `Array` для простоты примера представляет собой шаблон статического массива, размер которого задан статической константой `size{ 5 }`.

- Присваивание `arr[i] = T()` приводит к тому, что поля массива получают значения типа `T` по умолчанию. Для числовых типов это `0`, для `string` — пустая строка, для пользовательских типов — значение, которое задает для данного типа конструктор по умолчанию (конструктор без параметров).
- Члены-функции класса обеспечивают доступ к элементам массива, сортировку и вывод массива на экран.
- При обращении к элементам массива `T arr[size]` осуществляется проверка индекса.
- Функция `stopProgram` используется для аварийного завершения программы в случае ошибки.

Использование шаблона класса достаточно простое. При вызове конструктора класса обязательно указываем параметр-тип шаблона в угловых скобках

```
Array<int> intArray;
Array<string> strArray;
```

Далее просто вызываем нужные методы.

```
intArray.display();
int size = intArray.getSize();
strArray.sort();
```

В приведенной программе на основе шаблона класса `Array` мы создаем два объекта разных типов:

- массив целых `intArray`;
- массив строк `strArray`;

и с помощью методов шаблона выполняем над ними одинаковые действия — присвоение значений, сортировку, вывод на экран.

Результат работы программы приведен на рисунке 3.

```
Class Template Array

int Array initialization:
0 0 0 0 0

int Array after assignment:
5 4 3 2 1

int Array after ordering:
1 2 3 4 5

str Array initialization:

str Array after assignment:
two seven zero four one

str Array after ordering:
four one seven two zero

Press any key to exit program
```

Рисунок 3. Использование шаблона класса *Array* для работы с массивами разных типов

В приведенном выше примере мы использовали самый простой способ написания кода шаблона класса — все функции были размещены в теле класса. Но это не всегда удобно, например, бывает желательно отделить прототипы методов класса от их реализации.

Реализация метода вне тела класса требует полного описания типов, используемых в методе. При этом необходимы несколько ссылок на шаблон.

Пусть у нас есть шаблон класса `Sample` с методом `square`:

```
template<class T>
class Sample
{
    ...
    T square(T number);
}
```

Реализация метода `square` вне класса будет выглядеть так:

```
template<class T>
T Sample<T>::square(T number)
{
    T result = number * number;
    return result;
}
```

Во-первых, нужно указать перед заголовком функции, что это шаблон:

```
template<class T>
```

Во-вторых, нужно указать, что эта функция принадлежит шаблону класса `Sample` с параметром-типом `T`:

```
Sample<T>::square
```

Приведем пример программы с шаблоном класса `Array`, преобразованным таким образом (код программы — в папке `Lesson05\les05_03`):

Основные изменения по сравнению с предыдущей версией:

- код шаблона класса `Array` перенесен в отдельный заголовочный файл `array.h`;
- этот заголовочный файл включен в файл `les05_03.cpp`, в котором используется класс `Array`;

```
#include <iostream>
#include <conio.h>
#include "array.h"
using namespace std;

void stopProgram(string message)
{
    cout << message << endl;
    cout << "Press any key to exit program" << endl;
    _getch();
    exit(1);
}

int main()
{
    cout << "Class Tempate Array" << endl << endl;

    Array<int> intArray;
    . . .
```

- в файле `array.h` тело класса включает только поля и прототипы методов класса:

```
#pragma once
#include <iostream>

void stopProgram(std::string message);
template<class T>
```

```
class Array
{
    static const size_t size{ 5 };
    T arr[size];

public:
    Array();
    int getSize() const;
    T getItem(size_t index) const;
    void setItem(size_t index, T value);
    void display();
    void sort();
};
```

- реализация методов не включена в тело класса:

```
template<class T>
Array<T>::Array()
{
    for (int i = 0; i < size; i++)
    {
        arr[i] = T();
    }
}

template<class T>
int Array<T>::getSize() const
{
    return size;
}

template<class T>
T Array<T>::getItem(size_t index) const
{
    if (index >= 0 && index < size)
    {
```

```

        return arr[index];
    }
    else
    {
        stopProgram("Index is out of range!");
    }
}
. . .

```

Необходимо запомнить, что `template<class T>` и `Array<T>::` обязательно должны присутствовать в заголовке функции, реализованной вне тела класса. Мы знаем, что код шаблона функции следует помещать в заголовочном файле (`*.h`). То же относится и к шаблонам классов. Весь код шаблона класса (и объявление класса, и реализацию его методов) рекомендуется помещать в одном заголовочном файле.

Если размещение всего кода сделает файл `*.h` слишком большим или неупорядоченным, то альтернативой будет помещение реализации в файл `*.inl` (`.inl` от англ. «*inline*» = «встроенный»), а затем подключение `*.inl` к файлу `*.h`. Это даст тот же результат, что и размещение всего кода в заголовочном файле, но таким образом код будет физически разделенным.

Если реализация функций шаблона класса будет расположена в отдельном файле `*.cpp`, то в результате будут те же ошибки, что и при размещении шаблона функции в файле `*.cpp` — компиляция программы пройдет успешно, а вот сборка завершится фатальными ошибками, потому что функции шаблона класса просто не будут скомпилированы.

Параметрами шаблона класса (внутри угловых скобок) могут быть не только параметры-типы (`<class T>`), но и параметры обычных типов. Такие параметры называются **non-type** параметрами.

Рассмотрим пример (код программы — в папке *Lesson05\les05_04*).

```
#include <iostream>
#include <conio.h>
using namespace std;

template <class T, int size> // size – параметр non-type
                             // в шаблоне класса

class StatArray
{
private:
    // параметр non-type size определяет размер
    // статического массива
    T m_array[size];

public:
    T* getArray();
    T& operator[](int index)
    {
        return m_array[index];
    }
};

// определение метода вне тела класса с параметром non-type
template <class T, int size>

T* StatArray<T, size>::getArray()
{
    return m_array;
}
```

```

int main()
{
    cout << "Class Template with Non-type parameters"
          << endl << endl;

    // целочисленный массив из 10 элементов
    const int size = 10;
    StatArray<int, size> intArray;
    for (int count = 0; count < size; ++count)
        intArray[count] = count;

    // элементы массива в обратном порядке
    for (int count = size - 1; count >= 0; --count)
        std::cout << intArray[count] << " ";
    std::cout << '\n';

    // массив типа double из 8 элементов
    StatArray<double, 8> doubleArray;
    for (int count = 0; count < 8; ++count)
        doubleArray[count] = 5.5 + 0.1 * count;
    for (int count = 0; count < 8; ++count)
        std::cout << doubleArray[count] << ' ';

    _getch();
    return 0;
}

```

Результат работы программы приведен на рисунке 4.

```

Class Template with Non-type parameters

9 8 7 6 5 4 3 2 1 0
5.5 5.6 5.7 5.8 5.9 6 6.1 6.2

```

Рисунок 4. Применение *non-type* параметра шаблона класса

В этом примере параметр шаблона `size` (типа `int`) отвечает за размер выделяемого в классе статического массива.

```
template <class T, int size>
class StatArray
{
private:
    T m_array[size];
```

Этот параметр, наряду с другими, должен участвовать в каждом определении метода класса вне тела класса

```
template <class T, int size>
T* StatArray<T, size>::getArray()
{
    return m_array;
}
```

При создании объекта класса `non-type` параметр должен задаваться константным значением.

```
const int size = 10;
StatArray<int, size> intArray;

StatArray<double, 8> doubleArray;
```

В общем случае значением `non-type` параметра может быть константное выражением следующего вида:

- целочисленное значение или перечисление;
- указатель или ссылка на объект класса;
- указатель или ссылка на функцию;
- указатель или ссылка на метод класса.

Параметром шаблона класса может быть еще и — шаблон класса! Рассмотрим пример (код программы — в папке *Lesson05\les05_05*):

```
#include <iostream>
#include <vector>
#include <conio.h>
using namespace std;

template<class T>
struct Point
{
    T x;
    T y;
    void display()
    {
        cout << "(" << x << "," << y << ")";
    }
};

template<class T>
struct Array
{
    vector<T> data;
    void Add(T item)
    {
        data.push_back(item);
    }
    void display()
    {
        for(auto var:data)
            cout << var << " ";
    }
};

template<template<class> class T, class T1>
struct Some
{
```

```

T<T1> data; // создается переменная data,
            // типом которой будет шаблон класса T,
            // принимающий параметр-тип T1
void Add(T1 item)
{
    data.Add(item);
}
void display()
{
    data.display();
    cout << endl;
}
};

int main()
{
    // структура Point с целыми x,y
    Some<Point, int> intPoint;
    intPoint.data.x = 1;
    intPoint.data.y = 2;
    cout << "Some: struct Point with int x, y : ";
    intPoint.display();
    // структура Point с плавающими x,y
    Some<Point, double> doublePoint;
    doublePoint.data.x = 10.01;
    doublePoint.data.y = 0.02;
    cout << "Some: struct Point with double x,y : ";
    doublePoint.display();

    // массив (вектор) целых
    Some<Array, int> intArray;
    intArray.Add(1);
    intArray.Add(3);
    intArray.Add(5);
    cout << "Some: array (vector) with int items: ";
    intArray.display();
}

```

```

    _getch();
    return 0;
}

```

В программе имеется:

- шаблон класса **Point** (координаты точки) с параметром-типом **T** (тип координат)

```

template<class T>
struct Point
{
    T x;
    T y;
    . . .
}

```

- шаблон класса **Array** (массив) с параметром-типом **T** (тип элементов массива)

```

template<class T>
struct Array
{
    vector<T> data;
    . . .
}

```

- а также шаблон класса **Some** (нечто) с параметром-шаблоном класса **<template>class T** и параметром-типом **T1** для этого шаблона

```

template<template<class> class T, class T1>
struct Some
{
    T<T1> data;
    . . .
}

```

На основе шаблона `Some` создаются

```
// структура Point с целыми x,y
Some<Point, int> intPoint;
. . .

// структура Point с плавающими x,y
Some<Point, double> doublePoint;
. . .

// массив целых
Some<Array, int> intArray;
```

Пример, конечно, чисто учебный и практического смысла не имеет, но он показывает удивительные возможности шаблонов C++.

Результат работы программы приведен на рисунке 5.

```
Some: struct Point with int x, y : (1,2)
Some: struct Point with double x,y : (10.01,0.02)
Some: array (vector) with int items: 1 3 5
```

Рисунок 5. Использование параметров-шаблонов в шаблоне класса

Параметры шаблонов класса, как и обычные параметры, могут быть параметрами по умолчанию. Пусть мы зададим шаблон класса `Array` следующим образом:

```
template <class T = int, int size = 10>
class Array
```

Тогда следующее использование шаблона:

```
Array <> intArray;
```

будет равносильно такой записи:

```
Array <int, 10> intArray;
```

и приведет к созданию массива из 10 целых чисел.

Требования шаблона к используемым параметрам-типам. Стандартная библиотека шаблонов STL

Программный код шаблона может предъявлять определенные требования к используемым параметрам-типам.

Рассмотрим простой пример (код программы — в папке *Lesson05\les05_051*):

```
#include <iostream>
#include <conio.h>
using namespace std;

template< typename T >
void Display(T value)
{
    cout << value << endl;
}

template< typename T >
T Sum(T value1, T value2)
{
    return value1 + value2;
}

struct Point
{
    int x;
    int y;
};
```

```

int main()
{
    cout << "Template Requirements" << endl;

    int n1 = 1;
    int n2 = 2;
    int n3 = Sum(n1, n2);

    Display(n1);
    Display(n3);

    double d1 = 1.0;
    double d2 = 2.0;
    double d3 = Sum(d1, d2);

    Display(d1);
    Display(d3);

    Point p1{ 1,1 };
    Point p2{ 2,2 };
    // Point p3 = Sum(p1, p2);

    // Display(p1);
    // Display(p3);

    _getch();
    return 0;
}

```

В приведенном примере имеются:

- шаблон функции `void Display(T value)`, которая выводит значение параметра на экран;
- шаблон функции `T Sum(T value1, T value2)`, которая возвращает сумму значений двух своих параметров;
- структура `Point`, представляющая координаты (x, y) точки на плоскости;

- функция `main`, тестирующая шаблоны функций `Display` и `Sum` для параметров-типов `int`, `double` и `Point`.

Если раскомментировать закомментированные строки

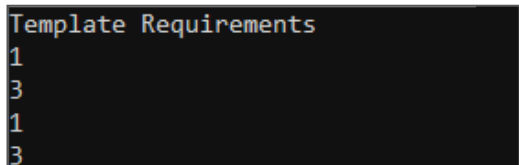
```
// Point p3 = Sum(p1, p2);  
  
// Display(p1);  
// Display(p3);
```

то компилятор выдаст сообщения об ошибках

```
Error C2676 binary '+': 'T' does not define this operator  
or a conversion to a type acceptable to the predefined  
operator
```

```
Error C2679 binary '<<': no operator found which takes  
a right-hand operand of type 'T' (or there is  
no acceptable conversion)
```

При закомментированных строках программа работает нормально (см. рис. 6)



```
Template Requirements  
1  
3  
1  
3
```

Рисунок 6. Использование шаблонов функций для разных параметров-типов

Следовательно, причина ошибок — не в шаблонах функций, а в том, что эти шаблоны требуют от

параметра-типа, в них участвующих (а это `Point`), соблюдения определенных правил поведения, а именно:

- в используемом типе должен быть определен (перегружен) оператор `+` (для шаблона функции `Sum`) ;
- в используемом типе должен быть определен (перегружен) оператор `<<` (для шаблона функции `Display`).

Добавим необходимые перегрузки в структуру `Point` (код программы — в папке `Lesson05\les05_051a`):

```
. . .
struct Point
{
    int x;
    int y;

    Point(int x = 0, int y = 0) : x{x}, y{y}
    {}

    Point operator+(Point n)
    {
        return Point(this->x + n.x, this->y + n.y);
    }

    friend ostream& operator<<(ostream& output, const Point& p)
    {
        output << "(" << p.x << "," << p.y << ")";
        return output;
    }
};

int main()
{
    cout << "Template Requirements" << endl;
    int n1 = 1;
```

```

int n2 = 2;
int n3 = Sum(n1, n2);

Display(n1);
Display(n3);

double d1 = 1.0;
double d2 = 2.0;
double d3 = Sum(d1, d2);

Display(d1);
Display(d3);

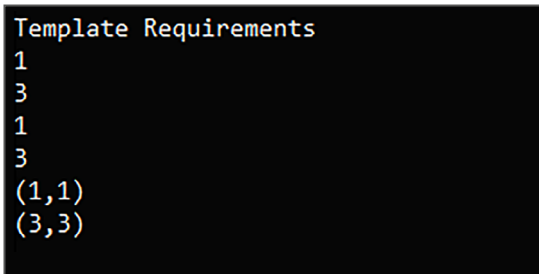
Point p1{ 1,1 };
Point p2{ 2,2 };
Point p3 = Sum(p1, p2);

Display(p1);
Display(p3);

_getch();
return 0;
}

```

Результат работы программы приведен на рисунке 7.



```

Template Requirements
1
3
1
3
(1,1)
(3,3)

```

Рисунок 7. Использование шаблонов функций для параметров-типов с перегруженными операторами

Мы видим, что требования шаблонов функций к параметрам-типам выполнены, и программа работает нормально для всех используемых типов данных.

Шаблоны класса, как правило, еще более чувствительны к используемым параметрам-типам. Кроме перегрузки операторов, (как в случае с шаблонами функций, шаблонам классов обычно требуется в используемых типах наличие следующих элементов:

- конструктор по умолчанию;
- конструктор копирования;
- оператор присваивания;
- операторы сравнения;
- операторы ввода — вывода.

Для некоторых шаблонов также могут потребоваться:

- деструктор;
- конструктор перемещения.

Также могут быть необходимы перегрузки и других (стандартных) методов, применяемых в коде шаблона, например, перегрузка методов `min`, `max`, `swap` и т. д.

Рассмотрим следующий пример. Возьмем шаблон класса `template<class T> class Array` из программы `Lesson05\les05_03` и попытаемся подставить в качестве параметра-типа `T` структуру `Point` (код программы — в папке `Lesson05\les05_052`)

```
struct Point
{
    int x;
    int y;
```

```

};

int main()
{
    cout << "Class Template Array (Points)" << endl << endl;

    Array<Point> pointArray;
    cout << "Point Array initialization:" << endl;
    pointArray.display();
    int size = pointArray.getSize();

    for (int i = size; i > 0; i--)
    {
        pointArray.setItem(size - i, Point{ i,i });
    }

    cout << endl << "Point Array after assignment:" << endl;
    pointArray.display();
    pointArray.sort();
    cout << endl << "Point Array after ordering:" << endl;
    pointArray.display();
    cout << endl;

    cout << endl << "Press any key to exit program" << endl;
    _getch();
    return 0;
}

```

При компиляции получаем ошибки:

C2679 binary '<<': no operator found which takes a right-hand operand of type 'T' (or there is no acceptable conversion)

C2676 binary '>': 'T' does not define this operator or a conversion to a type acceptable to the predefined operator

Компилятор сообщает, что в параметре-типе `T` (которым является `struct Point`) не определены оператор вывода `<<` и оператор сравнения `>`.

Определяем необходимые для использования шаблона `template<class T> class Array` операторы в классе `struct Point` (код программы — в папке *Lesson05\les05_052a*)

```
struct Point
{
    int x;
    int y;

    friend bool operator>(const Point& p1, const Point& p2)
    {
        double d1 = sqrt(p1.x * p1.x + p1.y * p1.y);
        double d2 = sqrt(p2.x * p2.x + p2.y * p2.y);
        return d1 - d2;
    }

    friend std::ostream& operator<<(std::ostream& output,
                                     const Point& p)
    {
        output << "(" << p.x << ", " << p.y << ")";
        return output;
    }
};
```

Теперь программа компилируется и работает правильно (см. рис. 8).

Шаблоны являются чрезвычайно полезным и удобным инструментом программирования и поэтому широко используются в C++. Множество готовых к использованию шаблонов (классов и функций) содержится в стандартной библиотеке шаблонов (Standard Template

Library — STL). Эта библиотека представляет собой неотъемлемую часть языка C++.

```

Class Template Array (Points)

Point Array initialization:
(0,0) (0,0) (0,0) (0,0) (0,0)

Point Array after assignment:
(5,5) (4,4) (3,3) (2,2) (1,1)

Point Array after ordering:
(1,1) (2,2) (3,3) (4,4) (5,5)

Press any key to exit program

```

Рисунок 8. Использование шаблона класса для параметров-типов с перегруженными операторами

STL содержит набор классов контейнеров для представления массивов, связанных списков, деревьев, наборов значений, наборов пар «значение — ключ» и т. д. STL также содержит функции обработки элементов этих классов (например, нахождение максимума, суммы, или отдельного элемента), и другие вспомогательные функции.

Одним из наиболее часто используемых шаблонов STL является **vector** (вектор, одномерный массив) — шаблон динамического массива. Объявляется вектор таким образом:

```
vector<тип_данного> имя_переменной;
```

Например, оператор

```
vector <int> intArray;
```

объявляет динамический массив целых чисел. Добавить данные в этот массив можно с помощью функций вектора:

- `push_back` — добавление значения после последнего элемента вектора;
- `insert` — вставка значения перед указанным элементом вектора.

Эти функции добавляют новые элементы и увеличивают размер вектора. Доступ к элементам вектора осуществляется так же, как к элементам обычного массива, например, `intArray[2] = 10`.

Для удаления используется функция `erase`, которая удаляет указанный элемент вектора и уменьшает размер вектора.

Таким образом, вектор представляет собой стандартный массив, размер которого может изменяться во время выполнения программы путем добавления и удаления его элементов.

Специализация шаблона функции

При использовании шаблонов функций мы получаем одну и ту же реализацию (исполнение одного и того же алгоритма) для разных типов данных. Иногда это не работает — вдруг оказывается, что для некоторых типов данных алгоритм должен быть иным.

Рассмотрим следующий пример (код программы — в папке `Lesson05\les05_06`):

```
#include <iostream>
#include <conio.h>
using namespace std;
```

```

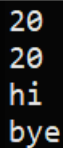
template <class T>
T Max(T t1, T t2)
{
    return (t1 > t2 ? t1 : t2);
}

int main()
{
    int i1 = 10;
    int i2 = 20;
    double d1 = 10.0;
    double d2 = 20.0;
    string s1 = "hi";
    string s2 = "bye";
    const char* p1 = "hi";
    const char* p2 = "bye";
    cout << Max(i1, i2) << endl;
    cout << Max(d1, d2) << endl;
    cout << Max(s1, s2) << endl;
    cout << Max(p1, p2) << endl;

    _getch();
    return 0;
}

```

Шаблон функции `T Max(T t1, T t2)` при использовании должен возвращать большее из значений своих двух параметров. Проверяем (см. рис. 9):



```

20
20
hi
bye

```

Рисунок 9. Использование шаблона функции *Max* для разных типов данных

С типами `int`, `double`, `string` функция работает правильно. А вот результат сравнения данных типа `const char*` — неверен: вместо правильного «*hi*» функция возвращает «*bye*». Все дело в том, что в этом случае сравниваются не значения данных, а значения адресов данных, значения указателей на массивы символов...

Получается, что для `const char*` нужна другая реализация шаблона функции. И такая возможность в C++ есть, этот механизм называется специализацией шаблона функции. Специализация шаблона существует для того, чтобы скорректировать поведение общего шаблона под особенности отдельных типов данных.

Специализация шаблона функции представляет собой такое определение, в котором за ключевым словом `template` следует пустая пара угловых скобок `<>`, а за ними — определение специализированного шаблона, в котором указывается имя шаблона, аргументы, для которых он специализируется, список параметров функции и ее тело.

```
template <>
returnDataType functionName <dataType> (argumentList)
{
    functionBody
}
```

Для приведенного выше шаблона функции `Max` специализация для типа данных `const char*` будет выглядеть следующим образом (код программы — в папке `Lesson05\les05_06a`):

```
template <>
const char* Max<const char*>(const char* t1, const char* t2)
{
    return (strcmp(t1, t2) > 0 ? t1 : t2);
}
```

Специализация шаблона должна быть размещена после общего шаблона.

```
template <class T>
T Max(T t1, T t2)
{
    return (t1 > t2 ? t1 : t2);
}

template <>
const char* Max<const char*>(const char* t1, const char* t2)
{
    return (strcmp(t1, t2) > 0 ? t1 : t2);
}

int main()
{
    . . .
```

Теперь программа правильно работает со всеми типами данных, включая и `const char*` (рис. 10).

```
20
20
hi
hi
```

Рисунок 10. Использование специализации шаблона функции `Max` для типа данных `const char*`

Приведем еще один пример специализации шаблона функции. На странице 15 был рассмотрен шаблон функции `getValue` для ввода данных с клавиатуры. Этот общий шаблон хорош для ввода числовых данных, но данные типа `string` можно обрабатывать посредством гораздо более простого кода.

Ниже приведено определение общего шаблона `getValue` и специализации шаблона для `string`.

```
#include <iostream>
#include <string>

template <class T>
void getValue(std::string prompt, T& value)
{
    std::cout << prompt;
    std::cin >> value;

    while (std::cin.fail())
    {
        std::cin.clear();
        std::cin.ignore(32767, '\n');
        std::cout << "Error... try again" << std::endl;
        std::cout << prompt;
        std::cin >> value;
    }

    std::string endLine;
    getline(std::cin, endLine);
}

template <>
inline void getValue<std::string>(std::string prompt,
    std::string& value)
{
```

```
std::cout << prompt;
getline(std::cin, value);
}
```

Специализация шаблона определена в заголовочном файле с ключевым словом **inline**.

Если специализация определяет inline-функцию, то ее определение должно находиться в заголовочном файле.

Если специализация определяет не inline-функцию, то в заголовочном файле должно находиться только ее объявление (прототип), а определение (тело функции) должно располагаться в одном из файлов реализации.

Полная специализация шаблона класса

С шаблонами классов может возникнуть та же проблема, что и с шаблонами функций:

- для некоторых типов данных возможна более эффективная реализация, чем та, которая определена в общем шаблоне;
- для некоторых типов данных вообще не подходит общая реализация.

Решается эта проблема также с помощью специализации шаблонов.

Полная (или явная) специализация шаблонов классов представляет собой определение шаблона класса, в котором после слова **template** указаны пустые угловые скобки **<>**, за которыми следует имя класса, конкретные параметры-типы класса и определение класса

```
template <> className<dataTypeList>{ classDefinition }
```

Явную специализацию шаблона класса можно определять только после того, как общий шаблон уже был объявлен (при этом может быть не определен). Иными словами, должно быть известно, что специализируемое имя обозначает шаблон класса.

Рассмотрим пример, в котором класс — шаблон `List` представляет собой список с возможностью добавления и удаления элементов (`add`, `remove`) и с функциями поиска минимального и максимального элементов (`getMin`, `getMax`).

Наиболее простым способом реализации таких возможностей является использование стандартных средств языка C++. Для организации динамического списка наиболее подходящим средством будет `vector` — шаблон класса «динамический массив». Для создания вектора необходимо вызвать конструктор шаблона, например, в классе `List` используется конструктор вектора по умолчанию `vector<T> list`; который создает пустой динамический массив данных типа `T`.

Для добавления элемента в этот массив в методе `add(T item)` используется функция вектора `pushback` — добавление элементов в конец вектора.

Для удаления элемента с указанным индексом из массива в методе `remove(int index)` используется функция вектора `erase` — удаление элемента, указанного посредством указателя.

Роль указателей при работе с вектором выполняют так называемые итераторы. Итератор — переменная специального типа, указывающая на элемент вектора.

Для указания на первый элемент вектора обычно используется такая конструкция: `имя_вектора.begin()`;

Указание на конец вектора (позиция за последним элементом) осуществляется таким образом: `имя_вектора.end()`;

Для прохождения по элементам вектора с итератором, как и с обычными указателями, можно выполнять арифметические действия.

- `list.begin() + index`; — это указатель на первый элемент вектора, смещенный на `index` элементов, то есть — указатель на элемент с номером `index`.
- `for (auto it = list.begin(); it < list.end(); it++)` — это цикл прохождения указателя `it` по всем элементам вектора, начиная с первого `it = list.begin()`; заканчивая последним `it < list.end()`; каждый раз переходя к следующему элементу `it++`.

Для получения элемента, на который указывает итератор, его можно разыменовать с использованием такого же синтаксиса, как и для обычных указателей (операция `*`).

`*it < *pMin` — элемент на который указывает итератор (указатель) `it` сравнивается с элементом, на который указывает указатель `pMin`.

`return *pMin;` — возвращает значение элемента, на которое указывает указатель `pMin`, т. е. минимальное.

Итак, класс `template<class T> class List` — динамический массив с возможностью поиска максимального и минимального элементов создан, и мы испытываем его с разными значениями параметра-типа `T` (код программы — в папке `Lesson05\les05_07`):

```
#include <iostream>
#include <vector>
#include <conio.h>
```

```
using namespace std;

template<class T>
class List
{
    vector<T> list;
public:
    void add(T item)
    {
        list.push_back(item);
    }

    void remove(int index)
    {
        list.erase(list.begin() + index);
    }

    auto getMin()
    {
        auto pMin = list.begin();
        for (auto it = list.begin(); it < list.end(); it++)
        {
            if (*it < *pMin)
            {
                pMin = it;
            }
        }
        return *pMin;
    }

    auto getMax()
    {
        auto pMax = list.begin();
        for (auto it = list.begin(); it < list.end(); it++)
        {
            if (*it > *pMax)
            {
```

```

        pMax = it;
    }
}
return *pMax;
}
};

int main()
{
    List<int> intList;
    intList.add(4);
    intList.add(5);
    intList.add(2);
    intList.add(3);
    cout << "intList min: " << intList.getMin() << endl;
    cout << "intList max: " << intList.getMax() << endl
         << endl;

    List<string> strList;
    strList.add("four");
    strList.add("five");
    strList.add("two");
    strList.add("three");
    cout << "strList min: " << strList.getMin() << endl;
    cout << "strList max: " << strList.getMax() << endl
         << endl;

    List<const char*> txtList;
    txtList.add("four");
    txtList.add("five");
    txtList.add("two");
    txtList.add("three");
    cout << "txtList min: " << txtList.getMin() << endl;
    cout << "txtList max: " << txtList.getMax() << endl
         << endl;
}

```

```

    _getch();
    return 0;
}

```

На рисунке 11 приведен результат работы программы.

```

intList min: 2
intList max: 5

strList min: five
strList max: two

txtList min: four
txtList max: three

```

Рисунок 11. Использование шаблона класса *List*

Из рисунка видно, что с числовыми (`int`) и строковыми (`string`) данными шаблон работает правильно, а вот текстовые константы (`const char*`) обрабатываются шаблоном неверно. Минимальное и максимальное значение списка `txtList` определяются не в лексикографическом порядке (по алфавиту), как это требуется, а просто берутся первое и последнее значения в списке, то есть в порядке адресов (указателей).

Таким, образом, для правильной работы с типом `const char*` нужно разработать специализированный шаблон класса *List*.

Рассмотрим определение полной специализации шаблона класса *List* (код программы — в папке *Lesson05/les05_07a*):

```

template<>
class List<const char*>
{
    vector<const char*> list;
public:
    void add(const char* item)
    {
        list.push_back(item);
    }

    void remove(int index)
    {
        list.erase(list.begin() + index);
    }

    auto getMin()
    {
        auto pMin = list.begin();
        for (auto it = list.begin(); it < list.end(); it++)
        {
            if (strcmp(*it, *pMin) < 0)
            {
                pMin = it;
            }
        }
        return *pMin;
    }

    auto getMax()
    {
        auto pMax = list.begin();
        for (auto it = list.begin(); it < list.end(); it++)
        {
            if (strcmp(*it, *pMax) > 0)
            {
                pMax = it;
            }
        }
    }
}

```

```

    }
    return *pMax;
}
};

```

Отличия полной специализации шаблона от общего шаблона:

- после ключевого слова `template` — пустые угловые скобки

```
template<>;
```

- после имени класса — значение параметра-типа в угловых скобках

```
class List<const char*>;
```

- в коде шаблона класса используется не имя параметра-типа (`T`), а имя типа данных

```
const char*.
```

На рисунке 12 приведен результат работы программы.

```

intList min: 2
intList max: 5

strList min: five
strList max: two

txtList min: five
txtList max: two

```

Рисунок 12. Использование полной специализации шаблона класса *List*

Из рисунка 12 видно, что теперь все типы данных обрабатываются правильно. Следует помнить, что в специализации шаблона должны быть определены все методы, которые необходимы для работы со специализируемым типом. Если, например, мы закомментируем метод `add` в специализации шаблона, то программа *les05_07a* не скомпилируется, а если закомментируем метод `remove`, то программа скомпилируется и будет прекрасно работать, поскольку метод `remove` в данной программе не вызывается...

В рассмотренном шаблоне класса `List` только один тип-параметр. Если в шаблоне класса имеется несколько параметров, то при полной специализации этого класса угловые скобки после `template` — пустые, а после имени класса в угловых скобках перечисляются все конкретные типы-параметры, например

```
template<>
class List<const char*, int>
{. . .}
```

Частичная специализация шаблона класса

Бывают ситуации, когда в шаблоне класса нужно специализировать не все типы-параметры, а только некоторые из них. Такая проблема решается посредством частичной специализации шаблона класса.

Если у шаблона класса есть несколько параметров, то можно специализировать его только для одного или нескольких аргументов, оставляя другие неспециализированными, то есть оставляя возможность задавать их значения при использовании шаблона.

Частичная специализация позволяет, таким образом, создать шаблон, соответствующий общему во всем, кроме тех параметров, вместо которых подставлены фактические типы или значения.

Частичная специализация может понадобиться при определении реализации, более подходящей для конкретного набора аргументов.

Рассмотрим пример (код программы — в папке *Lesson05/les05_08*):

```
#include <iostream>
#include <vector>
#include <conio.h>
using namespace std;

void stopProgram(string message)
{
    cout << message << endl;
    cout << "Press any key to exit program" << endl;
    _getch();
    exit(1);
}

template <class T, int height, int width>
class Matrix
{
    T m[height][width];
public:
    const auto& operator()(int j, int i) const
    {
        if (j < 0 || j >= height
            || i < 0 || i >= width)
        {
            stopProgram("Matrix index error!");
        }
    }
}
```

```

        return m[j][i];
    }

    auto& operator()(int j, int i)
    {
        if (j < 0 || j >= height
            || i < 0 || i >= width)
        {
            stopProgram("Matrix index error!");
        }
        return m[j][i];
    }

    auto getMin()
    {
        auto min = m[0][0];
        for (int j = 0; j < height; j++)
        {
            for (int i = 0; i < width; i++)
            {
                if (m[j][i] < min)
                {
                    min = m[j][i];
                }
            }
        }
        return min;
    }

    auto getMax()
    {
        auto max = m[0][0];
        for (int j = 0; j < height; j++)
        {
            for (int i = 0; i < width; i++)
            {

```

```

        if (m[j][i] > max)
        {
            max = m[j][i];
        }
    }
}
return max;
}
};

int main()
{
    const int height = 2;
    const int width = 3;

    Matrix<int, height, width> intMatrix;
    intMatrix(0, 0) = 5;
    intMatrix(0, 1) = 1;
    intMatrix(0, 2) = 9;
    intMatrix(1, 0) = 15;
    intMatrix(1, 1) = 11;
    intMatrix(1, 2) = 19;
    cout << "intMatrix min: " << intMatrix.getMin() << endl;
    cout << "intMatrix max: " << intMatrix.getMax()
        << endl << endl;

    Matrix<string, height, width> strMatrix;
    strMatrix(0, 0) = "five";
    strMatrix(0, 1) = "one";
    strMatrix(0, 2) = "nine";
    strMatrix(1, 0) = "fifteen";
    strMatrix(1, 1) = "eleven";
    strMatrix(1, 2) = "nineteen";
    cout << "strMatrix min: " << strMatrix.getMin() << endl;
    cout << "strMatrix max: " << strMatrix.getMax()
        << endl << endl;
    Matrix<const char*, height, width> txtMatrix;

```

```

txtMatrix(0, 0) = "five";
txtMatrix(0, 1) = "one";
txtMatrix(0, 2) = "nine";
txtMatrix(1, 0) = "fifteen";
txtMatrix(1, 1) = "eleven";
txtMatrix(1, 2) = "nineteen";
cout << "txtMatrix min: " << txtMatrix.getMin() << endl;
cout << "txtMatrix max: " << txtMatrix.getMax()
    << endl << endl;

_getch();
return 0;
}

```

Класс — шаблон `Matrix` представляет собой статический двумерный массив (матрицу) с возможностью изменения значений элементов (посредством перегруженного оператора `()`) и с функциями поиска минимального и максимального элементов (`getMin`, `getMax`).

Размеры матрицы задаются через `non-type` параметры класса `height`, `width`. Функция `main` тестирует работу `Matrix` с разными типами данных.

На рисунке 13 приведен результат работы программы.

```

intMatrix min: 1
intMatrix max: 19

strMatrix min: eleven
strMatrix max: one

txtMatrix min: five
txtMatrix max: nineteen

```

Рисунок 13. Использование шаблона класса `Matrix`

Из рисунка 13 можем увидеть, что с числовыми (`int`) и строковыми (`string`) данными шаблон работает правильно, а вот текстовые данные (`const char*`) обрабатываются шаблоном неверно. Минимальное и максимальное значение матрицы `txtMatrix` определяются не в лексикографическом порядке (по алфавиту), как это требуется, а просто берутся первое и последнее значения в двумерном массиве, то есть в порядке адресов (указателей).

Таким, образом, для правильной работы с типом `const char*` нужно разработать специализированный шаблон класса `Matrix`.

Но здесь возникает небольшая проблема. Параметрами шаблона матрицы являются и ее размеры. Получается, что для каждого нового сочетания размеров придется создавать отдельную специализацию шаблона класса, например:

```
template <>
class Matrix<const char*, 2, 3>

template <>
class Matrix<const char*, 3, 2>
```

и т. д.

Решением проблемы является создание частичной специализации, в которой специализируется только первый параметр шаблона (параметр-тип `T`).

Рассмотрим определение частичной специализации шаблона класса `Matrix` (код программы — в папке `Lesson05\les05_08a`):

```

template <int height, int width>
class Matrix<const char*, height, width>
{
    const char* m[height][width];
public:
    const auto& operator()(int j, int i) const
    {
        if (j < 0 || j >= height
            || i < 0 || i >= width)
        {
            stopProgram("Matrix index error!");
        }
        return m[j][i];
    }

    auto& operator()(int j, int i)
    {
        if (j < 0 || j >= height
            || i < 0 || i >= width)
        {
            stopProgram("Matrix index error!");
        }
        return m[j][i];
    }

    auto getMin()
    {
        auto min = m[0][0];
        for (int j = 0; j < height; j++)
        {
            for (int i = 0; i < width; i++)
            {
                if (strcmp(m[j][i], min) < 0)
                {
                    min = m[j][i];
                }
            }
        }
    }
}

```

```

    }
    return min;
}

auto getMax()
{
    auto max = m[0][0];
    for (int j = 0; j < height; j++)
    {
        for (int i = 0; i < width; i++)
        {
            if (strcmp(m[j][i], max) > 0)
            {
                max = m[j][i];
            }
        }
    }
    return max;
}
};

```

Отличия частичной специализации шаблона от полной специализации:

- после ключевого слова `template` в угловых скобках указываются неспециализированные параметры:

```
template <int height, int width>;
```

- после имени класса в угловых скобках указываются значения специализированных параметров (`const char*`) и имена неспециализированных параметров (`height, width`):

```
class Matrix<const char*, height, width>
```

На рисунке 14 приведен результат работы программы.

```
intMatrix min: 1
intMatrix max: 19

strMatrix min: eleven
strMatrix max: one

txtMatrix min: eleven
txtMatrix max: one
```

Рисунок 14. Использование частичной специализации шаблона класса *Matrix*

Из рисунка видно, что теперь все типы данных обрабатываются правильно.

Примеры создания шаблонов классов

Рассмотрим пример очень простого шаблона класса *Pair* (*Пара*).

Нам довольно часто приходится иметь дело со структурами, состоящими из двух элементов, например:

- `Point(int x, int y)` — координаты точки на экране;
- `Point(double x, double y)` — координаты точки на координатной плоскости;
- `Fraction(int x, int y)` — дробь;
- `Money(int dollar, int cent)` — денежная сумма;
- `Name(string firstName, string surname)` — имя и фамилия;
- `Mark(Name student, int mark)` — оценка ученика.

Наверное, было бы удобно иметь шаблон класса (структуры), который определил бы основные операции со всеми возможными структурами такого типа.

Рассмотрим шаблона класса `Pair` и его использование (код программы — в папке `Lesson05\les05_09`):

```
#include <iostream>
#include <conio.h>
using namespace std;

template <typename T1, typename T2>
struct Pair
{
    T1 first;
    T2 second;
    Pair() : first(T1()), second(T2()) {};
    Pair(const T1& x, const T2& y) : first{x}, second{y}{}

    void display()
    {
        cout << "(" << first << "," << second << ")";
    }

    bool operator==(const Pair& pair)
    {
        return this->first == pair.first &&
               this->second == pair.second;
    }

    bool operator!=(const Pair& pair)
    {
        return !(*this == pair);
    }
};

int main()
{
```

```

Pair<int, int>point1(0, 0);
Pair<int, int>point2(1, 1);
point1.display();
cout << " ";
point2.display();
cout << " ";
cout << (point1 == point2 ? "equals" : "not equals")
      << endl;

Pair<int, int> point;
point.display();
cout << " ";
point1.display();
cout << " ";
cout << (point != point1 ? "not equals" : "equals")
      << endl;

Pair<string, string> name("John", "Smith");
Pair <Pair<string, string>, int > mark(name, 8);
mark.first.display();
cout << " : " << mark.second << endl;
_getch();
return 0;
}

```

Шаблон структуры `Pair` имеет два параметра-типа `T1` и `T2`, определяющие соответственно типы первого и второго элементов пары.

Конструктор по умолчанию `Pair()` заносит значения по умолчанию типов `T1` и `T2`, в первый и второй элементы пары.

Конструктор с параметрами `Pair(const T1& x, const T2& y)` запоминает значения параметров.

Функция `display()` выводит значения структуры на экран.

Переопределенные операторы `==` и `!=` сравнивают значения структур.

Функция `main` тестирует шаблон класса для разных типов данных (см. рис. 15).

```
(0,0) (1,1) not equals
(0,0) (0,0) equals
(John,Smith) : 8
```

Рисунок 15. Использование шаблона класса *Pair*

Для создания пары `mark(name, 8)`, первым элементом которой является пара, требуется вложенное указание параметров-типов

```
Pair<string, string> name("John", "Smith");
Pair <Pair<string, string>, int > mark(name, 8);
```

Использовать функцию `display()` для переменной `mark` мы не можем, поскольку для `mark.first` (который имеет тип `<string, string>`) не определен оператор вывода `<<`. Но мы можем вывести данные этой структуры таким образом:

```
mark.first.display();
cout << " : " << mark.second << endl;
```

Рассмотрим еще один пример, — шаблон класса «Матрица (двумерный массив)» (код программы — в папке `Lesson05\les05_10`).

Код шаблона класса находится в файле *Matrix.h*:

```

#pragma once
#include <iostream>

void stopProgram(std::string message);

template <class T>
class Matrix
{
protected:
    int height;
    int width;
    T** m;

private:
    bool allocate(int height, int width)
    {
        if (height <= 0 && width <= 0)
        {
            return false;
        }

        this->height = height;
        this->width = width;
        m = new T * [height];
        for (int j = 0; j < height; j++)
        {
            m[j] = new T[width];
            for (int i = 0; i < width; i++)
            {
                m[j][i] = T();
            }
        }
        return true;
    }

    void clear()
    {

```

```

        if (m != nullptr)
        {
            for (int j = 0; j < height; j++)
            {
                delete[] m[j];
            }
            delete[] m;
        }
        m = nullptr;
    }

    void copyTo(T** from)
    {
        for (int j = 0; j < height; j++)
        {
            for (int i = 0; i < width; i++)
            {
                m[j][i] = from[j][i];
            }
        }
    }

public:
    Matrix(int height, int width)
    {
        if (height <= 0 || width <= 0)
        {
            stopProgram("Wrong sizes!");
        }

        if (!allocate(height, width))
        {
            stopProgram("Wrong sizes!");
        }
    }

```

```

Matrix(int size) : Matrix(size, size)
{}

Matrix()
{
    height = width = 0;
    m = nullptr;
}

Matrix(const Matrix& matrix)
{
    clear();
    allocate(matrix.height, matrix.width);
    copyTo(matrix.m);
}

~Matrix()
{
    clear();
}

auto& operator()(int j, int i)
{
    if (j < 0 || j >= height
        || i < 0 || i >= width)
    {
        stopProgram("Matrix index error!");
    }

    return m[j][i];
}

Matrix<T>& operator=(const Matrix<T>& matrix)
{
    if (&matrix == this)
    {
        return *this;
    }
}

```

```

        clear();
        allocate(matrix.height, matrix.width);
        copyTo(matrix.m);
        return *this;
    }

    friend std::ostream& operator<<(std::ostream & output,
        const Matrix& matrix)
    {
        if (matrix.m == nullptr)
        {
            output << "Empty matrix" << std::endl;
        }
        for (int j = 0; j < matrix.height; j++)
        {
            for (int i = 0; i < matrix.width; i++)
            {
                output << " " << matrix.m[j][i];
            }
            output << std::endl;
        }
        return output;
    }
};

```

`stopProgram` — прототип функции, которая аварийно завершает работу программы в случае ошибок. Тело функции определено в файле `les05_10.cpp`.

Объекты класса будут хранить двумерный динамический массив элементов типа `T`, поэтому полями класса являются:

- ссылка на двумерный динамический массив `T** m`;
- размеры массива `int height, int width` (высота и ширина матрицы).

В конструкторах, деструкторах и перегрузках операторов класса `Matrix` неоднократно используются операции выделения динамической памяти, освобождения этой памяти, копирования элементов, поэтому имеет смысл написать и использовать следующие внутренние, приватные функции:

- `bool allocate(int height, int width)` — выделения памяти для матрицы;
- `void clear()` — освобождение памяти матрицы;
- `void copyTo(T** from)` — копирование элементов матрицы.

Конструкторы класса:

- `Matrix(int height, int width)` — создание матрицы с размерами `height`, `width`. Использует для выделения памяти функцию `allocate`;
- `Matrix(int size)` — создание квадратной матрицы с размером `size`;
- `Matrix()` — создание пустой матрицы;
- `Matrix(const Matrix& matrix)` — конструктор копирования для корректного копирования матрицы. Использует функции `clear`, `allocate`, `copyTo` для освобождения, выделения памяти и копирования элементов матрицы.

Деструктор класса:

`~Matrix()` — использует функцию `clear`.

Для доступа к элементам матрицы используется перегруженный оператор `()`: `auto& operator()(int j, int i)` — для чтения и записи элементов матрицы.

Для операции присвоения матриц используется перегруженный оператор `=`: `Matrix<T>& operator=(const Matrix<T>& matrix)` — использует функции `clear`, `allocate`, `copyTo`.

Для вывода матрицы на экран используется перегруженный оператор `<<`

```
friend std::ostream& operator<<(std::ostream & output,
                                const Matrix& matrix)
```

Класс `Matrix` используется для работы с матрицами, элементами которых являются данные любого типа, например:

```
Matrix<int> intMatrix(3);
Matrix<string> strMatrix(2, 3);
Matrix<Point> pointMatrix(2, 3);
```

Набор операций при этом — минимальный: создание матрицы, заполнение данными, изменение данных, доступ к отдельным значениям, вывод на экран.

При желании можно добавить и другие операции общего назначения, например, поиск максимального и минимального значений, различные сортировки и поиски.

Ну, а если мы хотим занести числовые данные и выполнять специальные операции над числовыми матрицами, — сложение, вычитание, умножение?

Добавлять такие операции в шаблон класса `Matrix` не стоит — ведь они будут бессмысленны для всех нечисловых параметров-типов `T`. Можно использовать

специализацию шаблона, но, по-видимому, самым простым решением будет использование наследования.

Наследование является одним из основополагающих принципов ООП. В соответствии с ним, класс может использовать переменные и методы класса, от которого он наследуется, как свои собственные.

Наследование реализуется таким образом:

```
class derived_class : public base_class
{
    . . .
};
```

где `derived_class` — класс-наследник. Его называют: производный класс, дочерний класс, подкласс;

`base_class` — имя класса, от которого происходит наследование. Этот класс называется: базовый класс, родительский класс, суперкласс.

Дочерний класс может использовать любые переменные и функции базового класса, которые имеют модификаторы доступа `public` или `protected`.

Естественно, в дочернем классе определяются его собственные переменные и функции, — для этого он и создается.

Код шаблона класса `NumberMatrix` (числовая матрица) находится в файле `NumberMatrix.h`:

```
#pragma once
#include "matrix.h"

template <class T>
```

```

class NumberMatrix : public Matrix<T>
{
public:
    NumberMatrix(int height, int width) :
        Matrix<T>(height, width)
    {}
    NumberMatrix(int size) : NumberMatrix(size)
    {}
    void setRand(int max = 100, int min = 0)
    {
        for (int j = 0; j < this->height; j++)
        {
            for (int i = 0; i < this->width; i++)
            {
                this->m[j][i] = (T)(min + rand() %
                    (max - min + 1));
            }
        }
    }

    friend NumberMatrix<T> operator+
        (const NumberMatrix<T>& m1,
         const NumberMatrix<T>& m2)
    {
        if (m1.height != m2.height ||
            m1.width != m2.width)
        {
            stopProgram("Matrix sizes do not match!");
        }

        NumberMatrix<T> mRes(m1.height, m1.width);
        for (int j = 0; j < mRes.height; j++)
        {
            for (int i = 0; i < mRes.width; i++)
            {
                mRes.m[j][i] = m1.m[j][i] + m2.m[j][i];
            }
        }
    }
}

```

```

    }
    return mRes;
}

friend NumberMatrix<T> operator-
    (const NumberMatrix<T>& m1,
     const NumberMatrix<T>& m2)
{
    if (m1.height != m2.height ||
        m1.width != m2.width)
    {
        if (m1.height != m2.height ||
            m1.width != m2.width)
        {
            stopProgram("Matrix sizes do not match!");
        }
    }

    NumberMatrix<T> mRes(m1.height, m1.width);
    for (int j = 0; j < mRes.height; j++)
    {
        for (int i = 0; i < mRes.width; i++)
        {
            mRes.m[j][i] = m1.m[j][i] - m2.m[j][i];
        }
    }
    return mRes;
}

friend std::ostream& operator<<(std::ostream& output,
    const NumberMatrix& matrix)
{
    if (matrix.m == nullptr)
    {
        output << "Empty matrix" << std::endl;
    }
    for (int j = 0; j < matrix.height; j++)
    {

```

```

        for (int i = 0; i < matrix.width; i++)
        {
            output << " " << matrix.m[j][i];
        }
        output << std::endl;
    }

    return output;
}
};

```

Шаблон класса `NumberMatrix` является наследником шаблона класса `Matrix`. Это записывается следующим образом:

```

template <class T>
class NumberMatrix : public Matrix<T>

```

после имени базового класса в угловых скобках указывается параметр-тип `<T>`.

Полями класса `NumberMatrix` являются защищенные переменные-члены базового класса `Matrix`

```

protected:
    int height;
    int width;
    T** m;

```

Конструкторы `NumberMatrix` вызывают соответствующие конструкторы базового класса:

```

NumberMatrix(int height, int width) : Matrix<T>(height, width)
    NumberMatrix(int size) : NumberMatrix(size)

```

В классе есть метод `setRand(int max = 100, int min = 0)`, который заполняет матрицу случайными значениями.

Математические операции с числовыми матрицами позволяют выполнить перегруженные операторы `+` и `-`:

```
friend NumberMatrix<T> operator+(const NumberMatrix<T>&
                                  m1, const NumberMatrix<T>& m2)
friend NumberMatrix<T> operator-(const NumberMatrix<T>&
                                  m1, const NumberMatrix<T>& m2)
```

Объекты `NumberMatrix` выводятся на экран с помощью перегруженного оператора `<<`

```
friend std::ostream& operator<<(std::ostream& output,
                                const NumberMatrix& matrix)
```

В функции `main` (файл `les05_10.cpp`) тестируются шаблоны классов `Matrix` и `NumberMatrix`.

```
#include <conio.h>
#include "numbermatrix.h"
using namespace std;

struct Point
{
    int x;
    int y;

    Point(int x = 0, int y = 0) : x{x}, y{y}
    {}

    friend bool operator>(const Point& p1, const Point& p2)
    {
        double d1 = sqrt(p1.x * p1.x + p1.y * p1.y);
        double d2 = sqrt(p2.x * p2.x + p2.y * p2.y);
```

```

        return d1 - d2;
    }

    friend std::ostream& operator<<(std::ostream& output,
                                    const Point& p)
    {
        output << "(" << p.x << "," << p.y << ")";
        return output;
    }
};

void stopProgram(string message)
{
    cout << message << endl;
    cout << "Press any key to exit program" << endl;
    _getch();
    exit(1);
}

int main()
{
    cout << "Class Matrix" << endl << endl;

    Matrix<int> intMatrix(3);
    for (int j = 0; j < 3; j++)
    {
        for (int i = 0; i < 3; i++)
        {
            intMatrix(j, i) = j * 10 + i;
        }
    }
    cout << "int Matrix:" << endl;
    cout << intMatrix << endl;

    Matrix<int> intMatrixA = intMatrix;
    cout << "Copy int Matrix:" << endl;
    cout << intMatrixA << endl;
}

```

```

Matrix<string> strMatrix(2, 3);
strMatrix(0, 0) = "string00";
strMatrix(0, 1) = "string01";
strMatrix(0, 2) = "string02";
strMatrix(1, 0) = "string10";
strMatrix(1, 1) = "string11";
strMatrix(1, 2) = "string12";
cout << "string Matrix:" << endl;
cout << strMatrix << endl;

Matrix<Point> pointMatrix(2, 3);
pointMatrix(0, 0) = Point(0, 0);
pointMatrix(0, 1) = Point(0, 1);
pointMatrix(0, 2) = Point(0, 2);
pointMatrix(1, 0) = Point(1, 0);
pointMatrix(1, 1) = Point(1, 1);
pointMatrix(1, 2) = Point(1, 2);
cout << "point Matrix:" << endl;
cout << pointMatrix << endl;

NumberMatrix<double> numberMatrixA(2, 3);
numberMatrixA.setRand();
cout << "double Number MatrixA:" << endl;
cout << numberMatrixA << endl;

NumberMatrix<double> numberMatrixB(2, 3);
numberMatrixB.setRand();
cout << "double Number MatrixB:" << endl;
cout << numberMatrixB << endl;

NumberMatrix<double> numberMatrixC = numberMatrixA +
                                     numberMatrixB;
cout << "MatrixC = MatrixA + MatrixB:" << endl;
cout << numberMatrixC << endl;

_getch();
return 0;
}

```

Структура `Point` используется для проверки функционирования шаблона `Matrix` с пользовательскими (не встроенными) типами данных.

Функция `stopProgram` вызывается в классе `Matrix` для аварийного завершения программы в случае ошибок в данных.

В функции `main` создаются матрицы типа `Matrix` из элементов типа `int`, `string`, `Point`, проверяются функции доступа к элементам, копирования, вывода данных.

Также создаются матрицы типа `NumberMatrix` и проверяется функция сложения матриц.

Результат работы программы приведен на рисунке 16.

```
Class Matrix
int Matrix:
0 1 2
10 11 12
20 21 22
Copy int Matrix:
0 1 2
10 11 12
20 21 22
string Matrix:
string00 string01 string02
string10 string11 string12
point Matrix:
(0,0) (0,1) (0,2)
(1,0) (1,1) (1,2)
double Number MatrixA:
41 85 72
38 80 69
double Number MatrixB:
65 68 96
22 49 67
MatrixC = MatrixA + MatrixB:
106 153 168
60 129 136
```

Рисунок 16. Использование шаблонов классов `Matrix` и `NumberMatrix`

Приведенный пример показывает, что различные, на первый взгляд, не связанные между собой возможности C++ (шаблоны классов, перегрузка операторов, наследование) могут (и должны!) успешно комбинироваться для написания качественных программ.

3. Шаблоны с переменным числом аргументов

Мы знаем, что в C++ существуют функции с переменным числом параметров. Такие функции объявляются так же, как обычные функции, но вместо последнего параметра ставится многоточие:

```
тип_возвращаемого_значения имя_функции(список_параметров, ...)
```

Например (код программы — в папке *Lesson05\les05_11*):

```
#include <iostream>
#include <conio.h>
using namespace std;

int getSum(int count, int first, ...)
{
    int sum = 0;
    int* p = &first;
    while (count--)
    {
        sum += *p;
        p++;
        p++;
    }
    return sum;
}

int main()
{
    int sum = getSum(5, 1, 2, 3, 4, 5);
```

```

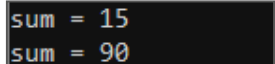
cout << "sum = " << sum << endl;
sum = getSum(4, 25, 0, 5, 60);
cout << "sum = " << sum << endl;

_getch();
return 0;
}

```

Первый параметр функции `getSum` — количество параметров в переменной части. Функция выбирает и суммирует значения всех этих параметров, зная, что они располагаются в памяти последовательно после параметра `first`, и что все эти параметры — целые числа.

Результат работы программы приведен на рисунке 17.



```

sum = 15
sum = 90

```

Рисунок 17. Использование функции с переменным числом параметров

Начиная с версии C++11 язык C++ предоставляет возможность также определять и шаблоны, принимающие переменное число параметров.

Шаблон с переменным числом параметров или вариативный шаблон (**Varadic Template**) — шаблон с неизвестным числом параметров. Синтаксис вариативного шаблона

```
template<class... Types>
```

где `Types` представляет собой пакет типов. В пакете типов могут присутствовать как параметры-типы, так и **non-type** параметры.

Пакет типов можно использовать в списке параметров функции:

```
template<class... Types>
void func(Types... parameters)
```

где `parameters` представляет собой пакет параметров, типы которых задаются указанными пакетом типов `Types`. Пакет параметров может быть только в функции-шаблоне, и он должен соответствовать пакету типов.

Внутри функции можно использовать как пакет типов, так и пакет параметров. Для этого после имени пакета ставится многоточие `...`, которое интерпретируется компилятором как раскрытие пакета в список (через запятую) в указанном месте. Обычным способом определения функций-шаблонов с переменным числом параметров является рекурсивное определение.

Рассмотрим пример (код программы — в папке `Lesson05\les05_12`):

```
#include <iostream>
#include <conio.h>

using namespace std;

double getSum(double x)
{
    return x;
}

template<class... Args>
double getSum(double x, Args... args)
{
```

```

    return x + getSum(args...);
}

int main()
{
    double sum = getSum(1, 2, 3, 4, 5);
    cout << "sum = " << sum << endl;

    sum = getSum(25, 0, 5, 60);
    cout << "sum = " << sum << endl;

    _getch();

    return 0;
}

```

Как это работает?

Вызов функции `getSum(25, 0, 5, 60)` на этапе компиляции приводит к созданию и вызову функции `getSum(25, ...)`, внутри которой будет оператор

```
return 25 + getSum(0, ...);
```

что, в свою очередь, приведет к созданию и вызову функции `getSum(0, ...)` и т.д...

Рекурсивный спуск (рекурсивный вызов `getSum` со все укорачивающимся списком параметров) прекратится, когда останется только 1 параметр и будет вызвана существующая обычная функция `double getSum(double x)`.

Во время работы программы все эти функции последовательно вызовут друг друга и таким образом просуммируют параметры `getSum(25, 0, 5, 60)`.

Результат работы программы приведен на рисунке 18.

```
sum = 15
sum = 90
```

Рисунок 18. Использование шаблона функции с переменным числом параметров

Рассмотрим еще один пример — аналог функции `print` (код программы — в папке *Lesson05\les05_13*):

```
#include <iostream>
#include <conio.h>
using namespace std;

void print()    // ничего не делает
{
}

template <class First, class... Other>
void print(const First& first, const Other&... other)
{
    cout << first;
    print(other...);
}

template <class... Args>
void println(const Args&... args)
{
    print(args...);
    cout << endl;
}

struct Point
{
    int x;
    int y;

    friend std::ostream& operator<<(std::ostream& output,
                                    const Point& p)
    {
```

```

        output << "(" << p.x << "," << p.y << ")";
        return output;
    }
};

int main()
{
    println("Hello, world");
    println("Pi = ", 3.14, '\n', 2, " * ", 2, " = ", 2 * 2);

    Point point{ 12,2 };
    println("Point: ", point);

    _getch();
    return 0;
}

```

Механизм работы этой программы такой же, как и в предыдущем примере: посредством рекурсии создается цепочка функций, каждая из которых выводит на экран по одному параметру.

Основная функция — `void print(const First& first, const Other&... other)`.

Функция `println` просто добавляет к `print` перевод на следующую строку.

Функция `void print()` нужна для завершения рекурсии.

Типы параметров могут быть любыми, лишь бы у них был перегружен оператор вывода `<<`.

В приведенном примере функция `println` выводит целые и плавающие числа, символы, массивы символов, значения структуры `Point`.

Результат работы программы приведен на рисунке 19.

```
Hello, world  
Pi = 3.14  
2 * 2 = 4  
Point: (12,2)
```

*Рисунок 19. Использование шаблона функции print
с переменным числом параметров*

4. Использование `std::initializer_list`

Для инициализации массивов (и статических, и динамических) удобно использовать список инициализации:

```
int array[] { 7, 6, 5, 4, 3, 2, 1 };  
int* dynArray = new int[7] { 7, 6, 5, 4, 3, 2, 1 };
```

Мы можем создать класс, который будет себя вести во всех отношениях практически как динамический массив (код программы — в папке *Lesson05\les05_14*):

```
class IntArray  
{  
private:  
    int length;  
    int* data;  
  
public:  
    IntArray() : length(0), data(nullptr)  
    {  
    }  
  
    IntArray(int length) : length(length)  
    {  
        data = new int[length];  
    }  
  
    ~IntArray()  
    {  
        delete[] data;  
    }  
}
```

```

int& operator[](int index)
{
    return data[index];
}

int getLength() const
{
    return length;
}
};

```

Но... инициализировать объект этого класса, как массив, мы не можем.

```

int main()
{
    IntArray array{ 7, 6, 5, 4, 3, 2, 1 }; // здесь –
                                           // ошибка компиляции
    for (int i = 0; i < 7; i++)
    {
        cout << array[i] << ' ';
    }
    cout << endl;
    _getch();
    return 0;
}

```

Строка `IntArray array{ 7, 6, 5, 4, 3, 2, 1 };` вызывает ошибку компиляции

```

Error    C2440    'initializing': cannot convert
from 'initializer list' to 'IntArray'

```

Компилятор C++ (начиная с версии C++11) автоматически конвертирует список инициализации в объект

типа `std::initializer_list`. Следовательно, для того чтобы объект класса мог инициализироваться списком, необходимо и достаточно в этом классе создать конструктор, который принимает в качестве параметра объект типа `std::initializer_list`.

Для работы с `std::initializer_list` нужно подключить заголовочный файл `<initializer_list>`. Класс `std::initializer_list` — это шаблон класса, поэтому мы должны указывать в угловых скобках после `std::initializer_list` какой тип данных будет использоваться, например, `std::initializer_list<int>` или `std::initializer_list<std::string>`. Для определения количества элементов в списке в `std::initializer_list` имеется функция `size()`.

Добавив в наш класс `IntArray` конструктор, который принимает тип данных `std::initializer_list` (код программы — в папке *Lesson05\les05_14a*):

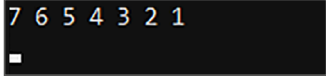
```
IntArray(const std::initializer_list<int>& list) :
    IntArray(list.size())
{
    int i = 0;
    for (auto& element : list)
    {
        data[i] = element;
        i++;
    }
}
```

Конструктор инициализации работает следующим образом:

- сначала создается массив `*data` размером `list.size()`, посредством вызова конструктора `IntArray(list.size())`,

- потом в цикле этот массив заполняется элементами из списка — параметра `initializer_list<int> list`.

Результат работы программы приведен на рисунке 20.



```
7 6 5 4 3 2 1
█
```

Рисунок 20. Использование типа `std::initializer_list` для создания конструктора со списком инициализации

Возможность использования списка инициализации таким же образом можно добавить и в шаблон класса.

Преобразуем наш класс `IntArray` в шаблон класса `DynArray` (код программы — в папке `Lesson05\les05_14b`):

```
#include <iostream>
#include <initializer_list>
#include <conio.h>
using namespace std;

template <class T>
class DynArray
{
private:
    int length;
    T* data;

public:
    DynArray() : length(0), data(nullptr)
    {
    }

    DynArray(int length) : length(length)
    {
        data = new T[length];
    }
}
```

```

DynArray(const std::initializer_list<T>& list) :
    DynArray(list.size())
{
    int i = 0;
    for (auto& element : list)
    {
        data[i] = element;
        i++;
    }
}

~DynArray()
{
    delete[] data;
}

T& operator[](int index)
{
    return data[index];
}

int getLength() const
{
    return length;
}
};

struct Point
{
    int x;
    int y;

    friend std::ostream& operator<<(std::ostream& output,
                                    const Point& p)
    {
        output << "(" << p.x << "," << p.y << ")";
        return output;
    }
};

```

```

    }
};

int main()
{
    DynArray<int> intArray{ 7, 6, 5, 4, 3, 2, 1 };
    for (int i = 0; i < 7; i++)
    {
        cout << intArray[i] << ' ';
    }
    cout << endl;

    DynArray<Point> pointArray{ Point{1,1}, Point{7, 7},
                               Point{3,3} };

    for (int i = 0; i < 3; i++)
    {
        cout << pointArray[i] << ' ';
    }
    cout << endl;

    _getch();
    return 0;
}

```

Теперь конструктор инициализации передает параметр-тип `T` шаблона класса `DynArray` в качестве параметра-типа шаблона класса `initializer_list`.

```

DynArray(const std::initializer_list<T>& list) :
    DynArray(list.size())

```

Все остальное — без изменений.

Результат работы программы приведен на рисунке 21.

```
7 6 5 4 3 2 1
(1,1) (7,7) (3,3)
```

Рисунок 21. Использование типа `std::initializer_list` для создания конструктора со списком инициализации в шаблоне класса

Список инициализации успешно используется и для массива целых чисел

```
DynArray<int> intArray{ 7, 6, 5, 4, 3, 2, 1 };
```

и для массива координат

```
DynArray<Point> pointArray{ Point{1,1}, Point{7, 7},
Point{3,3} }.
```

5. Класс string

Представление символов, таблицы кодировок

Компьютеры предназначены — для вычислений. Оперативная память хранит числа. Процессор обрабатывает числа. Текст... текст тоже хранится в виде набора чисел. Каждой букве соответствует числовой код. Программы работают с этими кодами и показывают их нам при необходимости в виде графических символов на экране или на бумаге. Коды объединяются в таблицы, которые называются кодировками. Кодировки бывают разные.

Разные — по количеству закодированных символов, по длине кода, по соответствию кодов — символам. Так сложилось исторически.

ASCII Code Chart																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Рисунок 22. Таблица ASCII

Самая первая, самая простая из стандартных кодировок — таблица **ASCII** (*American standard code for information interchange*). В этой таблице представлено все, что нужно англоязычному пользователю, — управляющие

символы, символы латинского алфавита, цифры и знаки препинания (см. рис. 22). Это кодировка с кодами от 0 до 127, в двоичном виде — от 00000000 до 11111111, в шестнадцатеричном — от 00 до 7F. Для хранения значений этого кода требуется 7 бит.

ASCII не поддерживает национальные алфавиты, в ней просто нет места для этих «излишеств». Поэтому было создано множество национальных восьмибитовых кодировок (коды от 0 до 255), в которых первые 127 символов совпадают с ASCII, а во второй половине таблицы (от 128 до 255) размещались символы других алфавитов, например, Windows-1250 — кодировка символов центрально-европейских языков, Windows-1251 — кодировка символов языков с кириллицей, Windows-1253 — кодировка символов греческого языка и т.д.

При использовании таких кодировок каждый символ занимает ровно 1 байт в памяти компьютера. Это удобно. Но очень неудобно работать с текстами, которым требуются одновременно несколько кодировок. Со временем, для решения этой проблемы создали одну большую общую таблицу с кодами для всех в мире символов из всех существующих алфавитов. Эта таблица — Unicode.

Заметьте, Unicode — это еще не кодировка. Кодировка характеризует:

- соответствие кодов и символов;
- способ размещения кодов в памяти компьютера.

Кодировке реализуется в программном обеспечении, работающем с этой кодировкой.

Unicode — это просто соглашение, просто придуманная таблица кодов символов. Она состоит из

1 114 112 позиций, которые сгруппированы в 17 блоков по 65536 символов. Нулевой блок — основной, он содержит символы всех современных алфавитов. Большинство блоков пока не заполнены.

Имея такой список кодов, можно придумать несколько различных способов размещения кодов в памяти, причем каждый из них будет по-своему эффективен. Эти способы называются формами представления (UTF — *Unicode Transformation Format*). Unicode имеет следующие формы представления:

- UTF-8;
- UTF-16;
- UTF-32.

UTF-8 — представление, которое обеспечивает наилучшую совместимость с прежними 8-битными кодировками. Символы UTF-8 с номерами меньше 128 занимают 1 байт и совпадают с символами ASCII. То есть текст, состоящих из кодов ASCII (английский текст, текст программы на C++, например...) — один и тот же и кодировке ASCII, и в кодировке UTF-8, и не требует конвертации.

Такое преимущество оборачивается определенными недостатками в других аспектах использования. Символы Unicode с номерами больше 128 в кодировке UTF-8 занимают от 2 до 6 байтов. То есть, кодировка UTF-8 является кодировкой переменной длины. Часто используемые символы (ASCII и национальных алфавитов) занимают 1 или 2 байта, редко используемые имеют большую длину. Таким образом, UTF-8 удобно применять для хранения текстов на внешних носителях для экономии пространства

и менее удобно — для обработки текстов в оперативной памяти из-за разной длины символов в байтах.

UTF-16 также является кодировкой переменной длины. Главное ее отличие от UTF-8 состоит в том, что базовой единицей в ней является не один, а два байта. То есть в кодировке UTF-16 любой символ Unicode может быть закодирован либо двумя, либо четырьмя байтами. Проще говоря, символы ASCII и национальных алфавитов (нулевой блок Unicode) записываются 2-мя байтами, более редкие символы — 4-мя байтами.

Практически, UTF-16 гораздо удобнее для обработки в оперативной памяти — все символы имеют длину в 2 байта. Ценой этого является дополнительное пространство, занимаемое ASCII-символами.

UTF-32 — кодировка Unicode, использующая для хранения любого символа ровно 32 бита. Символ UTF-32 является прямым представлением его кодовой позиции.

Главное преимущество UTF-32 перед кодировками переменной длины заключается в том, что символы UTF-32 непосредственно индексируемы. Получение n -ой позиции текста является операцией, занимающей одинаковое время. Напротив, коды с переменной длиной требуют последовательного доступа к n -ой позиции. Это делает замену символов в строках UTF-32 простой, для этого используется целое число в качестве индекса, как обычно делается для строк ASCII.

Главный недостаток UTF-32 — это неэффективное использование пространства, так как для хранения символа используется четыре байта. Символы, лежащие за пределами нулевой (базовой) плоскости кодового

пространства редко используются в большинстве текстов. Поэтому удвоение, в сравнении с UTF-16, занимаемого строками в UTF-32 пространства не оправдано.

Таким образом, мы видим, что в языках программирования:

- для обработки текстов, содержащих только латиницу удобно использовать кодировку ASCII или UTF-8;
- для обработки текстов, содержащих символы национальных алфавитов удобно использовать UTF-16 для обработки и, возможно, UTF-8 для сохранения на внешних носителях;
- UTF-32, возможно, имеет смысл использовать для обработки текстов, использующих нетрадиционные символы.

Символьные типы данных и строки C++

В языке C++ есть следующие (встроенные) типы данных для хранения кода символа:

```
char      ch{ 'a' };
char8_t   ch8{ u8'a' }; // требуется версия C++ 20
char16_t  ch16{ u'a' };
char32_t  ch32{ U'a' };
wchar_t   chw{ L'a' };   // нестандартный тип компилятора
                        // Microsoft
```

Тип `char` — исходный тип для хранения символов в C и C++. Этот тип можно использовать для хранения символов в кодировке ASCII или в любой восьмибитовой (однобайтной) кодировке, а также — для хранения отдельных байтов многобайтных символов Unicode.

`char` имеет разрядность 8 битов и представлен в двух модификациях `signed char` и `unsigned char`. Просто `char` обозначает `signed char`, то есть переменные такого типа (`char ch`) могут иметь значения в интервале от -128 до 127.

Тип `wchar_t` является определяемым реализацией типом расширенных символов. в компиляторе Microsoft он представляет 16-разрядный символ, используемый для хранения юникода в кодировке UTF-16.

Типы `char8_t`, `char16_t` и `char32_t` представляют 8-разрядные, 16-разрядные и 32-разрядные символы, соответственно (`char8_t` появился в c++ 20). Символы Unicode в кодировке UTF-8 могут храниться в типе `char8_t`. Символы Unicode в кодировке UTF-16 могут храниться в типе `char16_t`, а символы Unicode в кодировке UTF-32 могут храниться в типе `char32_t`.

Для хранения данных, представленных различными типами символов в C++, соответственно, имеются различные типы строк:

- `string` с элементами типа `char`;
- `wstring` с элементами типа `wchar_t`;
- `u16string` с элементами типа `char16_t`;
- `u32string` с элементами типа `char32_t`.

Все эти типы представляют собой специализации шаблона класса `basic_string` на основе соответствующих типов символов. Тип `string`, например, определен следующим образом:

```
typedef basic_string<char> string;
```

В шаблоне класса `basic_string`, таким образом, определены все необходимые функции для работы со строками разного типа. Рассмотрим их на примере класса `string`.

Цели и задачи класса `string`

Мы знаем, что для обработки текстовых данных в C++ можно использовать массив символов, в котором последний элемент имеет значение `'\0'` (нулевой символ).

```
char text[20]{ "This is a C-string" };
```

Такие массивы символов часто называют C-строками (Си-строками).

Массивы символов получили такое название в C++, потому что они являются наследием языка Си, в котором были единственным типом для хранения текстов.

А в C++ появился новый тип данных для обработки текстовой информации

```
string text{ "This is a C++ string" };
```

Это тоже строки, но уже C++-строки. Таким образом, в C++ есть два типа строк: C-строки и C++-строки.

Работа с Си-строками эффективна по скорости, но имеет существенные функциональные недостатки:

- их невозможно сравнивать посредством операторов сравнения (`==`, `<`, `>`);
- нельзя присваивать значения одного массива другому;
- хотя существует большой набор специальных функций для работы с Си-строками, не все из этих функций интуитивно понятны и удобны для использования.

Основным недостатком является невозможность динамического изменения размеров и опасность выхода за пределы массива при работе с ним. Для решения этих проблем был создан и включен в стандартную библиотеку класс `string`.

Преимущества класса `string` по сравнению с Си-строками:

- объект класса `string` является динамическим массивом, таким образом его легко редактировать;
- работа с этим объектом защищена от ошибок адресации;
- для работы с объектами `string` в классе существует большой набор полезных и удобных функций;
- класс `string` является контейнером (в языке C++ класс-контейнер — это класс, предназначенный для хранения и организации нескольких объектов определенного типа данных), поэтому для работы с объектами `string` можно использовать полный набор функций и операций, общий для всех контейнеров.

Анализ устройства класса `string`

Объявление, инициализация, доступ к элементам

Для работы с объектами класса `string` (далее будем называть их просто строками) достаточно подключить заголовок

```
#include <string>
```

Начиная с C++17, заголовочный файл `<string>` включен в заголовочный файл `<iostream>`, таким образом, в

консольных программах для работы со строками достаточно написать

```
#include <iostream>
```

`string` находится в пространстве имен `std`, поэтому для объявления переменной (без `using namespace std;`) нужно написать

```
std::string aString;
```

Объявление строки без ее инициализации создает пустую строку (значением `aString` после выполнения приведенной выше строки будет `" "`).

Если мы напишем так:

```
string text;
int number;

cout << "|" << number << "|" << text << "|" << endl;
```

то компилятор выдаст сообщение об ошибке только для переменной `number`:

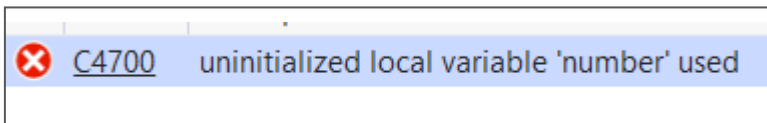


Рисунок 23

Целая переменная `int number` не имеет никакого значения.

Строковая же переменная `string text` имеет пустое значение.

Инициализировать строки можно различными способами:

- КОНСТАНТНЫМ МАССИВОМ СИМВОЛОВ

```
string text{ "Hello" }; // строка из const char*
```

- повторением символов (скобки — круглые!)

```
string name( 8, 'a' ); // строка из 8 'a' символов
```

- использованием оператора присвоения

```
string month = "March"; // то же, что string
                        // month{"March"};
```

В отличие от Си-строк, объекты `string` не содержат завершающий нулевой символ `'\0'`. Такой символ (`'\0'`) может присутствовать в строке на правах обычного символа, например

```
char stringC[]{ 'a', 'b', 'c', '\0', 'c', 'b', 'a' };
string stringCPP{ 'a', 'b', 'c', '\0', 'c', 'b', 'a' };
cout << " |" << stringC << "|" << endl;
cout << " |" << stringCPP << "|" << endl;
```

На экран будет выведено:

```
|abc|
|abc cba|
```

Рисунок 24

Мы видим, что символы Си-строки воспринимаются только до символа завершения, то есть, программа

понимает `stringC` как строку, состоящую из 3-х символов: `abc`. В C++ строке символ `"\0"` является таким же символом строки, как и все другие и отображается на экране как пустое место.

Длину строки можно определить с помощью методов `size()` или `length()`.

```
cout << text.size();
int len = name.length();
```

Доступ к отдельным символам строки осуществляется посредством оператора `[]`, как к элементам массива.

```
char ch = name[3];

for (int i = 0; i < name.length(); i++)
{
    cout << name[i];
}

for (auto c : name)
{
    cout << c;
}
```

Ввод и вывод

Вывод строки типа `string` ничем не отличается от вывода встроенных типов данных.

```
string text{ "Hello" };
cout << text << endl;
```

Для завершения вывода строки обычно используется манипулятор `endl`.

Вводить строку, содержащую пробелы, с помощью `cin` нельзя, так как ввод прекратится после встречи первого пробельного символа. К пробельным символам относятся: символы пробела ' ', табуляции '\t', перевода строки '\n', а также '\v', '\r', '\f'.

Для ввода строки, включающей пробельные символы, необходимо использовать функцию `getline()`. Синтаксис этой функции:

```
getline(istream, str, delim);
```

где `istream` — это потоковый объект (для ввода с клавиатуры используется `cin`), `str` — объект класса `string`, куда будет помещена строка, а `delim` — это символ — разделитель, определяющий конец текста, который будет введен в строку. `delim` — необязательный аргумент, по умолчанию используется '\n'.

Пример ввода строки

```
string text;
getline(cin, text);
```

Операторы

Для объектов класса `string` определены следующие операторы:

- присваивания — `=` ;
- ввода-вывода — `<<`, `>>` ;
- конкатенации (сложения строк) — `+`, `+=` ;
- сравнения — `==`, `!=`, `<`, `>`, `<=`, `>=` ;
- индексирования — `[]`.

Рассмотрим следующий фрагмент кода и вывод результатов на экран

```
string s1;
string s2;
string s3;

s1 = "abc";
s2 = { 'x', 'y', 'z' };
s3 = s1;

cout << " " << s1 << " " << s2 << " " << s3 << endl;
cout << " " << (s1 == s2) << " " << (s1 != s2) << " "
    << (s1 < s2) << " " << (s1 >= s2) << endl;
cout << " " << s1 + s2 << endl;
s2 += s1;
cout << " " << s2 << endl;
```

```
abc xyz abc
0 1 1 0
abcxyz
xyzabc
```

Рисунок 25

Переменной типа `string` можно присваивать значения других таких же переменных, значения Си-строк, значения массивов символов.

Строки можно сравнивать посредством операций «равно», «не равно», «больше-меньше».

Строки сравниваются в лексикографическом порядке, т.е. — как бы они располагались в словаре. Та строка, которая в словаре должна стоять выше, является «меньшей».

Операция сложения строк (конкатенация) просто добавляет строку, которая стоит после знака `+` к строке, стоящей до знака.

Методы класса `string`

Все методы класса `string` можно разделить на следующие группы:

- методы доступа к элементам;
- методы-итераторы;
- методы поиска;
- методы редактирования;
- методы работы со вместимостью;
- сервисные функции (не члены класса).

Методы доступа к элементам

Мы знаем, что доступ к отдельным символам строки можно получить по индексу (`text[i]`).

В дополнение к этому, можно получить специальный доступ к элементам строки с помощью методов доступа.

Таблица 1

Метод	Описание
<code>at()</code>	Получение указанного символа с проверкой выхода индекса за границы
<code>front()</code>	Получение первого символа (возвращается ссылка на символ)
<code>back()</code>	Получение последнего символа (возвращается ссылка на символ)
<code>data()</code>	Возвращает константный указатель на лежащий в основе строки C-массив, такой, что <code>data() + i == &operator[](i)</code> для каждого <code>i</code> в <code>[0, size() - 1]</code>

Метод at()

```
(1) char& at (size_t pos);
(2) const char& at (size_t pos) const;
```

Метод `at()` работает так же, как и доступ через оператор `[]`. Различие в том, что при выходе индекса за границы строки оператор `[]` никак на это не прореагирует, а метод `at()` сообщит об ошибке (выбросит исключение), что делает программу более защищенной от ошибок.

Вариант (1) — полный доступ к строке, вариант (2) — только чтение.

```
string name{ "12345" };
char c2 = name.at(10);
char c1 = name[10];
```

Метод front()

```
(1) char& front();
(2) const char& front() const;
```

Метод back()

```
(1) char& back ();
(2) const char& back () const;
```

Методы `front()` и `back()` возвращают ссылки на первый и последний символы строки, соответственно. Варианты — полный доступ и только чтение.

```
string name{ "12345" };
char& beg = name.front();
char& fin = name.back();
```

Метод *data()*

```
const char* data () const;
```

Метод *data()* позволяет работать со строкой как с С-строкой, правда, без возможности изменения ее.

```
string name{ "12345" };
const char* pname = name.data();
```

Методы-итераторы

Методы — итераторы возвращают указатели на элементы строки.

Таблица 2

Метод	Описание
<i>begin()</i> , <i>cbegin()</i>	Возвращает итератор на первый элемент
<i>end()</i> , <i>cend()</i>	Возвращает итератор на элемент, следующий за последним
<i>rbegin()</i> , <i>rend()</i> <i>crbegin()</i> , <i>crend()</i>	Возвращают реверсивные итераторы (для прохождения по строке в обратном порядке)

Пример использования:

```
string text{ "this is a c++ string" };
for (auto it = text.begin(); it < text.end(); it++)
{
    if (!isalpha(*it) && !isspace(*it))
    {
```

```

        cout << *it;
    }
}

cout << endl;

```



Рисунок 26

В приведенном фрагменте кода просматриваются все символы строки — от первого до последнего и выводятся на экран все не-буквы и не-пробелы, то есть знаки ++.

Методы поиска

Методы поиска позволяют найти в строке позицию искомой строки или символа.

Таблица 3

Метод	Описание
<code>find()</code>	Находит первую подстроку, равную переданной последовательности символов. Поиск начинается с заданной позиции
<code>rfind()</code>	Находит последнюю подстроку, равную переданной символьной последовательности. Поиск начинается с заданной позиции
<code>find_first_of()</code>	Находит первый символ строки, равный одному из символов в переданной последовательности символов. Поиск начинается с заданной позиции
<code>find_last_of()</code>	Находит последний символ, равный одному из символов в переданной последовательности символов. Поиск начинается с заданной позиции

Метод	Описание
<code>find_first_not_of()</code>	Находит первый символ строки, не равный ни одному из символов в переданной последовательности символов. Поиск начинается с заданной позиции
<code>find_last_not_of()</code>	Находит последний символ, не равный ни одному из символов в переданной последовательности символов. Поиск начинается с заданной позиции

Метод `find()`

```
(1) size_t find (const string& str, size_t pos = 0) const;
(2) size_t find (const char* s, size_t pos = 0) const;
(3) size_t find (const char* s, size_t pos, size_t n) const;
(4) size_t find (char c, size_t pos = 0) const;;
```

Метод `rfind()`

```
(1) size_t find (const string& str, size_t pos = 0) const;
(2) size_t find (const char* s, size_t pos = 0) const;
(3) size_t find (const char* s, size_t pos, size_t n) const;
(4) size_t find (char c, size_t pos = 0) const;
```

Метод `find()` возвращает позицию первого символа найденной подстроки или символа. По умолчанию поиск начинается с начала строки (с символа с номером `0`), но можно в параметре `pos` указать номер символа строки, с которого надо начинать поиск. Если подстрока не найдена, метод `find` возвращает значение `string::npos`.

Третий параметр `n`, если он присутствует, обозначает количество сравниваемых символов.

Метод `rfind` осуществляет поиск с конца строки. Параметр `pos`, если он есть, указывает номер символа, с которого осуществляется поиск по направлению к началу строки.

Рассмотрим фрагмент кода и результаты на экране

```
string text{ "This is a c++ string" };
cout << " " << text << endl;
cout << " " << text.find("is") << endl;      // 1 – find (2)
cout << " " << text.find("is", 4) << endl;    // 2 – find (2)
cout << " " << text.rfind("is") << endl;      // 3 – rfind (2)
cout << " " << text.rfind("is", 4) << endl;    // 4 – rfind (2)
cout << " " << text.find("strong", 0, 3) << endl; // 5 –
                                                // find (4)
cout << " " << text.find("strong") << endl;    // 6 – find (4)
```

```
This is a c++ string
2
5
5
2
14
4294967295
```

Рисунок 27

Первый поиск находит «*is*» в слове «*This*» (позиция 2), второй поиск пропускает это слово и находит «*is*» дальше (позиция 5), третий и четвертый — выполняются с конца строки — к началу (5 и 2 соответственно). Пятый поиск находит 3 буквы слова «*strong*» (позиция 14), шестой — не находит слово «*strong*» (на экран выводится значение `string::npos`).

Метод `find_first_of()`

```
(1) size_t find_first_of (const string& str, size_t pos = 0)
    const;
(2) size_t find_first_of (const char* s, size_t pos = 0)
    const;
(3) size_t find_first_of (const char* s, size_t pos, size_t n)
    const;
(4) size_t find_first_of (char c, size_t pos = 0) const;;
```

Метод ищет не подстроку, а любой из символов, заданных в этой подстроке.

Метод `find_last_of()` полностью аналогичен методу `find_first_of()`, но только выполняет поиск с конца строки.

Рассмотрим маленький пример:

```
string text{ "This is a c++ string" };
cout << " " << text << endl;
cout << " " << text.find_first_of("bac") << endl;    // 1
cout << " " << text.find_last_of("bac") << endl;      // 2
```

```
This is a c++ string
8
10
```

Рисунок 28

Первый поиск находит символ «a» (из набора символов «abc» (позиция 8)). Второй поиск (с конца) находит символ «c» (из набора символов «abc» (позиция 10)). Методы `find_first_not_of()` и `find_last_not_of()` полностью аналогичны методам `find_first_of()` и `find_last_of()`, но только ищут любой из символов, не входящих в подстроку.

Пример:

```
string text{ "this is a c++ string" };
cout << " " << text << endl;
cout << " "
    << text.find_first_not_of("abcdefghijklmnopqrstuvwxyz")
    << endl;
```

```
this is a c++ string
11
```

Рисунок 29

Поиск вернул позицию первой НЕ-буквы (символ +).

Методы редактирования

Методы редактирования используются для изменения строк.

Таблица 4

Метод	Описание
<code>assign()</code>	Заменяет содержимое строки
<code>clear()</code>	Удаляет все символы строки
<code>erase()</code>	Удаляет указанные символы из строки
<code>insert()</code>	Вставляет символы в строку
<code>push_back()</code>	Добавляет символ к концу строки
<code>pop_back()</code>	Удаляет последний символ строки
<code>append()</code>	Добавляет символы в конец строки
<code>replace()</code>	Заменяет часть строки на указанную подстроку
<code>substr()</code>	Возвращает подстроку
<code>copy()</code>	Копирует подстроку
<code>resize()</code>	Изменяет размер строки
<code>swap()</code>	Обменивает содержимое строки с другой строкой

Метод `assign()`

```
(1) string& assign (const string& str);
(2) string& assign (const string& str, size_t subpos,
                  size_t sublen);
(3) string& assign (const char* s);
(4) string& assign (const char* s, size_t n);
(5) string& assign (size_t n, char c);
```

Строка, к которой применяется `assign()` заменяется на:

- 1) строку `str`;
- 2) подстроку строки `str` (указывается начальная позиция `subpos` и число символов подстроки `sublen`);
- 3) Си-строку `s`;
- 4) подстроку Си-строки `s` (указывается число символов подстроки `n`);
- 5) заданное число повторений `n` указанного символа `c`.

Пример:

```
string text{ "this is a c++ string" };
string str{ "next string" };
cout << " " << text.assign(str) << endl;
cout << " " << text.assign(str, 3, 4) << endl;
cout << " " << text.assign("abcdef") << endl;
cout << " " << text.assign("abcdef", 2) << endl;
cout << " " << text.assign(5, 'a') << endl;
```

```
next string
t st
abcdef
ab
aaaaa
```

Рисунок 30

Метод *clear()*

```
void clear ();
```

Удаляет содержимое строки, которая становится пустой строкой с длиной 0.

Метод *erase()*

```
(1) string& erase (size_t pos = 0, size_t len = npos);
(2) iterator erase (const_iterator p);
(3) iterator erase (const_iterator first, const_iterator last);
```

Удаляет часть строки:

- 1) с указанной позиции *pos* заданное число символов *len*. По умолчанию удаляется вся строка;
- 2) удаляет символ, на который указывает итератор *p*;
- 3) удаляет символы в указанном диапазоне итераторов *first* — *last*.

Примеры:

```
string text{ "this is a c++ string" };
string str = text;
cout << " " << "|" << str.erase() << "|" << endl;

str = text;
cout << " " << "|" << str.erase(3, 10) << "|" << endl;

str = text;
str.erase(str.begin() + 1);
cout << " " << "|" << str << "|" << endl;

str = text;
str.erase(str.begin() + 1, str.end() - 3);
cout << " " << "|" << str << "|" << endl;
```

```

||
|thi string|
|tis is a c++ string|
|ting|

```

Рисунок 31

Метод insert()

```

(1) string& insert (size_t pos, const string& str);
(2) string& insert (size_t pos, const string& str,
                    size_t subpos, size_t sublen = npos);
(3) string& insert (size_t pos, const char* s);
(4) string& insert (size_t pos, const char* s, size_t n);
(5) string& insert (size_t pos, size_t n, char c);
(6) iterator insert (const_iterator p, char c);

```

Вставляет в строку с указанной позиции **pos**:

- 1) заданную строку **str**;
- 2) подстроку строки **str** (указывается начальная позиция **subpos** и число символов подстроки **sublen**);
- 3) Си-строку **s**;
- 4) подстроку Си-строки **s** (указывается число символов подстроки **n**);
- 5) заданное число **n** повторений указанного символа **c**.
- 6) указанный символ **c** в позицию по итератору **p**.

Примеры:

```

string text{ "this is a c++ string" };
string theStr{ "abcdef" };

string str = text;

```

```
cout << " " << str.insert(5, theStr) << endl;

str = text;
cout << " " << str.insert(5, theStr, 2, 3) << endl;

str = text;
str.insert(str.begin() + 2, 'A');
cout << " " << str << endl;
```

```
this abcdefis a c++ string
this cdeis a c++ string
thAis is a c++ string
```

Рисунок 32

Метод *push_back()*

```
void push_back(char c);
```

Добавляет символ *c* после последнего символа строки.

```
text.push_back('F');
```

Метод *pop_back()*

```
void pop_back();
```

Удаляет последний символ строки. Если строка пустая, поведение функции не определено, но исключение не возникает.

```
text.pop_back();
```

Метод `append()`

```

(1) string& append (const string& str);
(2) string& append (const string& str, size_t subpos,
                    size_t sublen = npos);
(3) string& append (const char* s);
(4) string& append (const char* s, size_t n);
(5) string& append (size_t n, char c);

```

Добавляет в конец строки указанные символы.

Перегрузки метода `append` аналогичны перегрузкам метода `insert`.

Метод `replace()`

```

(1) string& replace (size_t pos, size_t len,
                    const string& str);
    string& replace (const_iterator i1, const_iterator i2,
                    const string& str);
(2) string& replace (size_t pos, size_t len,
                    const string& str, size_t subpos,
                    size_t sublen = npos);
(3) string& replace (size_t pos, size_t len, const char* s);
    string& replace (const_iterator i1, const_iterator i2,
                    const char* s);
(4) string& replace (size_t pos, size_t len,
                    const char* s, size_t n);
    string& replace (const_iterator i1, const_iterator i2,
                    const char* s, size_t n);
(5) string& replace (size_t pos, size_t len, size_t n,
                    char c);
    string& replace (const_iterator i1, const_iterator i2,
                    size_t n, char c);

```

Заменяет часть строки на заданную строку `str`.

Часть строки, которую надо заменить, указывается:

- либо парой: начальная позиция `pos`, длина `len`,
- либо парой указателей (итераторов) `i1`, `i2` на начало и конец подстроки.

Строка для замены может быть:

- 1) строкой `str`;
- 2) подстрокой `str` (с позиции `subpos`, длиной `sublen`);
- 3) Си-строкой `*s`;
- 4) подстрокой Си-строки `*s` длиной `n`;
- 5) строкой, созданной из символа `c`, повторенного `n` раз.

Примеры:

```
// 01234567890123456789
string text{ "this is a c++ string" };
string str{ "abcdef" };

string txt = text;
txt.replace(10, 3, str);
cout << " " << txt << endl;

txt = text;
txt.replace(10, 3, str, 2, 2);
cout << " " << txt << endl;

txt = text;
txt.replace(10, 3, "new");
cout << " " << txt << endl;

txt = text;
txt.replace(10, 3, "1234", 1, 2);
cout << " " << txt << endl;

txt = text;
txt.replace(10, 3, 3, '!');
cout << " " << txt << endl;
```

```
txt = text;
txt.replace(txt.begin() + 10, txt.end() - 7, str);
cout << " " << txt << endl;
```

```
this is a abcdef string
this is a cd string
this is a new string
this is a 23 string
this is a !!! string
this is a abcdef string
```

Рисунок 33

Метод substr()

```
string substr (size_t pos = 0, size_t len = npos) const;
```

Возвращает новую строку — подстроку данной строки. Подстрока задается начальной позицией **pos** и длиной **len**. Если длина не задана, берется подстрока до конца строки. Исходная строка не изменяется.

Примеры:

```
// 01234567890123456789
string text{ "this is a c++ string" };

cout << " " << text.substr(5, 9) << endl;
cout << " " << text.substr(5) << endl;
cout << " " << text << endl;
```

```
is a c++
is a c++ string
this is a c++ string
```

Рисунок 34

Метод copy()

```
size_t copy (char* s, size_t len, size_t pos = 0) const;
```

Копирует подстроку данной строки с позиции `pos` длиной `len` в массив символов `*s`. Символ завершения Си-строки (`'\0'`) при этом не копируется.

Пример:

```
// 01234567890123456789
string text{ "this is a c++ string" };

char carr[10]{ 0 };
text.copy(carr, 8, 5);

cout << " " << carr << endl;
```



is a c++

Рисунок 35

Метод resize()

```
(1) void resize (size_t n);
(2) void resize (size_t n, char c);
```

Устанавливает размер строки равным заданному числу `n`. Если `n` меньше текущего размера, строка обрезается до заданного размера. Если же `n` больше текущего размера, строка увеличивается до заданного размера. При этом, если символ `c` задан, строка дополняется необходимым количеством `c`, если символ `c` не задан, строка дополняется нулевыми символами (`'\0'`).

Пример:

```
string str{ "abcdef" };
str.resize(10, '*');
cout << " " << str << endl;
```

```
abcdef****
```

Рисунок 36

Метод `swap()`

```
void swap (string& str);
```

Обменивает строку с другой строкой.

Пример:

```
string text{ "this is a c++ string" };
string str{ "abcdef" };
text.swap(str);
cout << " " << text << endl;
cout << " " << str << endl;
```

```
abcdef
this is a c++ string
```

Рисунок 37

Методы работы со вместимостью

Кроме длины, у строк есть еще одна характеристика размера — емкость (*capacity*). Дело в том, что при создании и изменении строки ей выделяется память не точно по количеству символов, которые будут заполнять строку,

а «с запасом». Это делается для того, чтобы уменьшить число запросов на выделение динамической памяти при небольшом увеличении размера строки.

Методы работы со вместимостью изменяют длину и емкость строк без изменения их содержимого.

Таблица 5

Метод	Описание
<code>empty()</code>	Проверяет, является ли строка пустой
<code>size()</code> , <code>length()</code>	Возвращает количество символов в строке
<code>max_size()</code>	Возвращает максимальное количество элементов, которое может содержать строка
<code>reserve()</code>	Задаёт емкость строки, минимум <code>size()</code> . Новая память для хранения выделяется при необходимости
<code>capacity()</code>	Возвращает количество символов, под которые у строки есть выделенное место.
<code>shrink_to_fit()</code>	Запрос для уменьшения емкости <code>capacity()</code> до <code>size()</code>

Метод `empty()`

```
bool empty() const;
```

Возвращает `true` если строка пустая (длина строки равно 0).

```
if (str.empty())
{
    cout << "No content" << endl;
}
```

Метод `size()`

```
size_t size() const;
```

Метод *length()*

```
bool length() const;
```

Возвращают длину строки (число символов в строке).

Метод *capacity()*

```
size_t capacity() const;
```

Возвращает емкость строки — число байтов, которое реально отведено под строку.

Метод *max_size()*

```
size_t max_size() const;
```

Возвращает максимально возможную длину строки.

Метод *reserve()*

```
void reserve (size_t n = 0);
```

Функция задает новую емкость строки *n*. Если *n* больше текущей емкости, емкость увеличивается до *n* или больше. Если *n* меньше текущей емкости, то уменьшение выполняется таким образом, чтобы длина строки осталась прежней.

Метод *shrink_to_fit()*

```
void shrink_to_fit();
```

Функция сокращает емкость строки, делая ее равной длине строки. На самом деле емкость после выполнения функции может остаться больше длины строки из соображений оптимизации работы с памятью.

Примеры:

```
string text{ "this is a c++ string" };
string str{ "abcdef" };
string none;
cout << " " << text.length() << endl;
cout << " " << text.size() << endl;
cout << " " << text.capacity() << endl;
cout << " " << text.max_size() << endl;

cout << " " << str.length() << endl;
cout << " " << str.capacity() << endl;
cout << " " << str.max_size() << endl;

cout << " " << none.length() << endl;
cout << " " << none.capacity() << endl;

text.reserve(8);
cout << text << endl;
cout << " " << text.capacity() << endl;

text.shrink_to_fit();
cout << " " << text.capacity() << endl;
```

```
20
20
31
2147483647
6
15
2147483647
0
15
this is a c++ string
31
31
```

Рисунок 38

Сервисные функции (не члены класса)

Сервисные функции предназначены для преобразований строковых данных и для чтения строк с пробельными символами.

Таблица 6

Метод	Описание
<code>getline()</code>	Считывает данные из потока до разделителя или до предельного количества символов.
<code>stoi(), stol(), stoll()</code>	Преобразует строку в целое (<code>int</code>), длинное целое (<code>long</code>), «сверхдлинное» целое (<code>long long</code>)
<code>stoul(), stoull()</code>	Преобразует строку в беззнаковое длинное целое (<code>long</code> , <code>long long</code>)
<code>stof(), stod(), stold()</code>	Преобразует строку в число с плавающей точкой (<code>float</code> , <code>double</code> , <code>long double</code>)
<code>to_string()</code>	Преобразует целое или число с плавающей точкой в строку

Метод `getline()`

```
(1) istream& getline (istream& is, string& str, char delim);
(2) istream& getline (istream& is, string& str);
```

Считывает строку из входного потока `is` в строковую переменную `str`. Чтение производится до символа-разделителя `delim` во входном потоке. Если разделитель не указан (по умолчанию), разделитель — `'\n'` (символ перевода строки).

Пример:

```
string text;
getline(cin, text);
```

Методы `stoi()`, `stol()`, `stoll()`

```
int stoi (const string& str, size_t* idx = 0,
          int base = 10);

long stol (const string& str, size_t* idx = 0,
           int base = 10);

long long stoll (const string& str, size_t* idx = 0,
                 int base = 10);
```

Эти функции преобразуют строку `str` в целое число.

Если указатель `idx` не равен нулю, функция помещает по этому адресу позицию в строке, которая соответствует первому нецифровому символу.

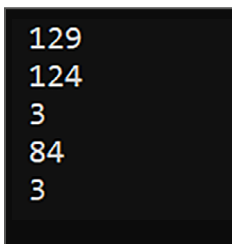
Параметр `base` по умолчанию определяет стандартную десятичную систему счисления. При необходимости можно указать другую систему счисления.

Примеры:

```
string text1{ "129" };
int number1 = stoi(text1);
cout << " " << number1 << endl;

unsigned int pos;
string text2{ "124,8" };
int number2 = stoi(text2, &pos);
cout << " " << number2 << endl;
cout << " " << pos << endl;

string text3{ "1248" };
int number3 = stoi(text3, &pos, 8);
cout << " " << number3 << endl;
cout << " " << pos << endl;
```



```
129
124
3
84
3
```

Рисунок 39

В первом примере строка «129» преобразуется в число 129.

Во втором примере в строке «124,8» символ с номером 3 (запятая) — это уже не цифра, поэтому на экран выдается полученное число 124 и позиция, на которой число заканчивается — 3.

В третьем примере параметр 8 указывает, что будет обрабатываться число в восьмеричной системе счисления. Поэтому цифра 8 — это уже не цифра (не восьмеричная цифра), а 124 в восьмеричной системе представляет десятичное число 84 ($1 * 64 + 2 * 8 + 4 = 84$).

Что мы и видим на экране.

Метод `stoul()`, `stoull()`

```
unsigned long stoul (const string& str, size_t* idx = 0,
                    int base = 10);

unsigned long long stoull (const string& str,
                           size_t* idx = 0, int base = 10);
```

Эти функции преобразуют строку `str` в положительные (`unsigned`) целые числа. Параметры функций аналогичны рассмотренным выше.

Пример:

```
unsigned int number1 = stoul(text1);
unsigned long number2 = stoul(text2);
```

Метод *stof()*, *stod()*, *stold()*

```
float stof (const string& str, size_t* idx = 0);
double stod (const string& str, size_t* idx = 0);
long double stold (const string& str, size_t* idx = 0);
```

Эти функции преобразуют строку *str* в числа с плавающей запятой. Параметры функций аналогичны рассмотренным выше.

Пример:

```
string orbits("365.24 29.53");
unsigned int sz;

double earthTurn = stod(orbits, &sz);
double moonTurn = stod(orbits.substr(sz));
cout << "The moon completes " << (earthTurn / moonTurn)
     << " orbits per Earth year" << endl;
```

```
The moon completes 12.3684 orbits per Earth year
```

Рисунок 40

В приведенном примере сначала преобразуется в плавающее число первая часть строки «365.24 29.53», затем вторая часть (подстрока от конца первой части до конца строки), затем вычисляется, сколько раз Луна вращается вокруг Земли за год.

Метод to_string()

```
(1) string to_string (int val);
(2) string to_string (long val);
(3) string to_string (long long val);
(4) string to_string (unsigned val);
(5) string to_string (unsigned long val);
(6) string to_string (unsigned long long val);
(7) string to_string (float val);
(8) string to_string (double val);
(9) string to_string (long double val);
```

Преобразует числа в разных представлениях — в строку.

Пример:

```
string piIs = "Pi is " + to_string(3.1415926);
cout << " " << piIs << endl;
```

```
Pi is 3.141593
```

Рисунок 41

Примеры использования класса string

Программа поиска в тексте

Код программы — в папке *Lesson05\les05_15*. В функции `main` инициализируется (строфой из стихотворения Киплинга) переменная `text`. В этом тексте ищется слово (подстрока) «*and*» — сначала с учетом регистра, а потом — без учета регистра.

```

#include <iostream>
#include <iomanip>
#include <conio.h>
using namespace std;

template <typename T>
void display(T* items, int size);
int countAll(const string where, const string what);
int* findAll(string where, string what, int& count);
string toUpr(const string text);
int* findAllR(const string where, const string what, int&
count);

int main()
{
    string text
    { R"(
        If you can keep your head when all about you
        Are losing theirsand blaming it on you;
        If you can trust yourself when all men doubt you,
        But make allowance for their doubting too :
        If you can wait and not be tired by waiting,
        Or, being lied about, don't deal in lies,
        Or being hated don't give way to hating,
        And yet don't look too good, nor talk too wise;
    )" };

    cout << "In text" << endl;
    cout << text << endl;

    cout << "Search for word \"and\"" << endl;
    cout << endl;

    cout << "case sensitive" << endl;
    int count;
    int* pos = findAll(text, "and", count);
    display(pos, count);
}

```

```

cout << endl;
cout << "case insensitive" << endl;
pos = findAllR(text, "and", count);
display(pos, count);

_getch();
return 0;
}

```

Результат работы программы приведен на рисунке 42.

In text

```

If you can keep your head when all about you
Are losing theirsand blaming it on you;
If you can trust yourself when all men doubt you,
But make allowance for their doubting too :
If you can wait and not be tired by waiting,
Or, being lied about, don't deal in lies,
Or being hated don't give way to hating,
And yet don't look too good, nor talk too wise;

```

Search for word "and"

```

case sensitive
    63      196

```

```

case insensitive
    63      196      308

```

Рисунок 42. Поиск подстроки в тексте

На экран выдаются списки позиций всех найденных слов.

В случае поиска с учетом регистра — это конец слова «*theirsand*» во второй строке и слово «*and*» в пятой строке.

В поиске без учета регистра добавляется еще слово «*And*» в последней строке.

Поиск выполняется функцией `findAll` (с учетом регистра) и функцией `findAllR` (без учета регистра). Обе эти функции возвращают указатель на массив найденных позиций `int* pos`, который выдается на экран функцией `display`.

Функция `findAll` получает параметры: где искать — `string where`, что искать — `string what`, возвращает указатель на массив найденных позиций и число найденных позиций `int& count`.

```
int* findAll(string where, string what, int& count)
{
    count = countAll(where, what);
    if (count == 0)
    {
        return nullptr;
    }

    int* array = new int[count];
    int pos = -1;
    for (int i = 0; i < count; i++)
    {
        pos = where.find(what, pos + 1);
        array[i] = pos;
    }

    return array;
}
```

Функция сначала определяет число найденных позиций посредством функции `countAll`, затем создает массив нужного размера и выполняет поиск слова в цикле.

Каждый раз поиск возобновляется с позиции, следующей за найденной (`pos + 1`). Для первого поиска с 0-й позиции `pos` устанавливается в `-1`. Если искомого слов в тексте нет, функция возвращает `nullptr`.

Функция `countAll` считает, сколько раз в тексте встречается искомое слово

```
int countAll(const string where, const string what)
{
    int count = 0;
    int pos = where.find(what);
    while (pos != string::npos)
    {
        count++;
        pos = where.find(what, pos + 1);
    }

    return count;
}
```

Поиск начинается с начала текста и выполняется каждый раз со следующей позиции.

Шаблон функции `display` используется для вывода на экран найденных позиций слова в тексте.

```
template <typename T>
void display(T* items, int size)
{
    for (int i = 0; i < size; i++)
    {
        cout << setw(8) << items[i];
    }
    cout << endl;
}
```

Функция `findAllR` имеет те же параметры, что и `findAll` и использует `findAll` для поиска.

```
int* findAllR(const string where, const string what,
              int& count)
{
    return findAll(toUpr(where), toUpr(what), count);
}
```

При этом и текст, в котором нужно искать, и иско-
мое слов преобразуются к верхнему регистру с помощью
функции `toUpr`. Таким образом различия в регистре от-
брасываются. Функция `toUpr` получает исходную строку
и возвращает ее в верхнем регистре.

```
string toUpr(const string text)
{
    string uText = text;

    for (int i = 0; i < uText.size(); i++)
    {
        uText[i] = toupper(uText[i]);
    }
    return uText;
}
```

Создается копия исходной строки и в ней каждый
символ приводится к верхнему регистру.

Программа создания частотного словаря текста

Код программы — в папке `Lesson05\les05_16`.

В функции `main` инициализируется переменная `text`
и создается словарь этого текста с указанием повторения

каждого слова. Понятно, что для создания реального словаря программа должна быть много сложнее и понимать грамматику, но в первом приближении мы эти тонкости учитывать не будем.

```
#include <iostream>
#include <iomanip>
#include <conio.h>
#include <vector>
#include <algorithm>

using namespace std;

struct Entry
{
    string word;
    int count;
};

vector<Entry> getDictionary(string text);
vector<string> getWords(string text);

int main()
{
    string text
    { R"(
        If you can keep your head when all about you
        Are losing theirsand blaming it on you;
        If you can trust yourself when all men doubt you,
        But make allowance for their doubting too :
        If you can wait and not be tired by waiting,
        Or, being lied about, don't deal in lies,
        Or being hated don't give way to hating,
        And yet don't look too good, nor talk too wise;
    )" };

    cout << "From text" << endl;
```

```

cout << text << endl;
cout << "Create dictionary" << endl;
cout << endl;
vector<Entry> dictionary = getDictionary(text);

for (auto entry : dictionary)
{
    cout << setw(12) << entry.word << setw(4)
        << entry.count << endl;
}

cout << endl;
_getch();

return 0;
}

```

Частотный словарь представляет собой массив структур **Entry** (слово, число повторений).

Создается словарь функцией **getDictionary**.

```

vector<Entry> getDictionary(string text)
{
    vector<Entry> entries;
    vector<string> words = getWords(text);
    sort(words.begin(), words.begin() + words.size(),
        sortByWords);
    int i = 0;

    while (i < words.size())
    {
        Entry entry{ words[i], 0 };
        while (i < words.size() && words[i] == entry.word)
        {
            entry.count++;
            i++;
        }
    }
}

```

```

    }
    entries.push_back(entry);
}

return entries;
}

```

В этой функции создается вектор всех слов в тексте (`vector<string> words`) посредством функции `getWords`, сортируется по алфавиту и затем переписывается в вектор словаря (`vector<Entry> entries`). При этом каждое слово переписывается только один раз и заносится количество повторений слова.

Сортировка вектора слов выполняется библиотечной функцией `sort`, в которую в качестве параметра передается функция сравнения слов `sortByWords`.

```

bool sortByWords(const string& word1, const string& word2)
{
    return word1 < word2;
}

```

Функция `getWords` формирует из текста набор всех входящих в этот текст слов.

```

vector<string> getWords(string text)
{
    vector<string> words;
    int pos = 0;

    while (pos < text.size())
    {
        while (pos < text.size() && !isalpha(text[pos]))

```

```

    {
        pos++;
    }
    if (pos == text.size())
    {
        break;
    }

    int wordStart = pos;

    while (pos < text.size() && isalpha(text[pos]))
    {
        pos++;
    }
    int wordLen = pos - wordStart;
    words.push_back(text.substr(wordStart, wordLen));
}
return words;
}

```

Переменная `pos` содержит текущую позицию текста и проходит по всем символам текста. В цикле, который ограничен размерами текста (`pos < text.size()`) каждый раз:

- находится начало следующего слова (или конец текста) — пропускаются все символы — не буквы;
- если конец слова — выход;
- запоминается начало слова;
- находится конец слова или конец текста — пропускаются все символы — буквы;
- формируется слово и добавляется в вектор слов.

Сформированный вектор слов возвращается в вызывающую функцию. Фрагмент результата работы программы приведен на рисунке 43.

From text

If you can keep your head when all about you
 Are losing theirsand blaming it on you;
 If you can trust yourself when all men doubt you,
 But make allowance for their doubting too :
 If you can wait and not be tired by waiting,
 Or, being lied about, don't deal in lies,
 Or being hated don't give way to hating,
 And yet don't look too good, nor talk too wise;

Create dictionary

And	1
Are	1
But	1
If	3
Or	2
about	2
all	2
allowance	1
and	1
be	1
being	2
blaming	1
by	1
can	3
deal	1

Рисунок 43. Создание частотного словаря

Можно было бы предварительно перевести весь текст в нижний регистр (см. предыдущую программу), тогда результаты были бы точнее.

6. Резюме

Раздел 1. Статический полиморфизм и шаблоны как частный случай

- Полиморфным является код, который во время исполнения программы может обрабатывать значения разных типов.
Имеется два типа полиморфизма:
 - ▷ полиморфизм времени выполнения (или динамический полиморфизм);
 - ▷ полиморфизм времени компиляции (или статический полиморфизм).
- Динамический полиморфизм реализуется во время работы программы. Он представляет собой использование виртуальных функций.
- Статический полиморфизм реализуется во время компиляции программы. C++ предоставляет три вида статического полиморфизма:
 - ▷ перегрузка функции;
 - ▷ перегрузка оператора;
 - ▷ шаблоны. Существуют шаблоны функций и шаблоны классов.
- Компилятор по типу параметра определяет, для какого типа нужно сгенерировать функцию. Но в некоторых случаях может возникнуть неоднозначность, и тогда указание параметра-типа при вызове шаблонной функции необходимо.
- Важной особенностью шаблона функции является то, что пока нет вызова функции, шаблон не компилируется.

ется. Шаблон компилируется в новую версию функции для каждого параметра-типа, для которого он вызывается.

- Тело шаблона функции необходимо размещать в файле-заголовке (в `H` — файле, а не в `CPP` — файле).
- Использование полиморфизма во всех его разновидностях делает программы более ясными и лучше сопровождаемыми. Применение шаблонов `C++`, в дополнение к этому, приводит к более коротким и легче изменяемым программам.
- Шаблоны широко используются в стандартных библиотеках `C++`, а написание современных программ немыслимо без активного применения этих библиотек.

Раздел 2. Шаблоны классов

- В языке `C++` обобщенное программирование основывается на шаблонах функций и шаблонах классов.
- Шаблоны классов позволяют создавать описание абстрактного класса, которое при использовании шаблона разворачивается компилятором в конкретные классы, реализующие заданные функции для различных типов данных.
- Шаблон класса начинается с ключевого слова `template`, за которым в угловых скобках следует список параметров шаблона класса.
- Параметрами шаблонов класса могут быть параметры-типы, параметры обычных типов, параметры-шаблоны.
- Код класса-шаблона ничем не отличается от обычного класса: поля класса, конструктор, методы класса,

только вместо конкретного указания типа данных используется имя параметра-типа шаблона (T).

- При вызове конструктора класса обязательно указываем параметр-тип шаблона в угловых скобках

```
Array<int> intArray;
```

- Далее просто вызываем нужные методы.

```
intArray.display();
```

- Реализация метода вне тела класса требует полного описания типов, используемых в методе.

```
template<class T>
T Sample<T>::square(T number)
{ ...
```

- Указание `template<class T>` и `Sample<T>` — обязательно. Весь код шаблона класса (и объявление класса, и реализацию его методов) рекомендуется помещать в одном заголовочном файле.
- Параметрами шаблона могут быть и параметры обычных типов — `non-type` параметры, например

```
template <class T, int size>
class StatArray
```

- Параметрами шаблона могут быть шаблоны классов, например

```
template<template<class> class T, class T1>
struct Some
```

- Параметры шаблонов класса, как и обычные параметры, могут быть параметрами по умолчанию, например

```
template <class T = int, int size = 10>  
class Array
```

- Программный код шаблона может предъявлять определенные требования к используемым параметрам-типам. В используемом типе должны быть перегружены операторы и функции, встречающиеся в коде шаблона.
- Шаблонам классов обычно требуется в используемых типах наличие следующих элементов:
 - ▷ конструктор по умолчанию;
 - ▷ конструктор копирования;
 - ▷ оператор присваивания;
 - ▷ операторы сравнения;
 - ▷ операторы ввода — вывода.
- Множество готовых к использованию шаблонов (классов и функций) содержится в стандартной библиотеке шаблонов (*Standard Template Library* — STL). Эта библиотека представляет собой неотъемлемую часть языка C++.
- Одним из наиболее часто используемых шаблонов STL является `vector` (вектор, одномерный массив) — шаблон динамического массива.
- Отдельное, специальное определение шаблона функции для конкретного типа данных называется специализацией шаблона. Специализация шаблона существу-

ет для того, чтобы скорректировать поведение общего шаблона под особенности отдельных типов данных.

- Специализация шаблона функции представляет собой такое определение, в котором за ключевым словом `template` следует пустая пара угловых скобок `<>`, а за ними — определение специализированного шаблона, в котором указывается имя шаблона, аргументы, для которых он специализируется, список параметров функции и ее тело.

```
template<>returnDataType functionName<dataType>(argumentList)
    {functionBody}
```

- Специализация шаблона должна быть размещена после общего шаблона.
- Если специализация определяет `inline`-функцию, то ее определение должно находиться в заголовочном файле.
- Если специализация определяет не `inline`-функцию, то в заголовочном файле должно находиться только ее объявление (прототип), а определение (тело функции) должно располагаться в одном из файлов реализации.
- Полная (или явная) специализация шаблонов классов представляет собой определение шаблона класса, в котором после слова `template` указаны пустые угловые скобки `<>`, за которыми следует имя класса, конкретные параметры-типы класса и определение класса

```
template <> className<dataTypeList>{ classDefinition }
```

- Если у шаблона класса есть несколько параметров, то можно специализировать его только для одного или нескольких аргументов, оставляя другие неспециализированными, то есть оставляя возможность задавать их значения при использовании шаблона. Такая специализация называется частичной специализацией шаблона класса.

Раздел 3. Шаблоны с переменным числом аргументов

- В C++ существуют функции с переменным числом параметров. Такие функции объявляются так же, как обычные функции, но вместо последнего параметра ставится многоточие:

```
тип_возвращаемого_значения имя_функции(список_параметров,
                                          ...)
```

- Язык C++ предоставляет возможность также определять и шаблоны, принимающие переменное число параметров. Шаблон с переменным числом параметров или вариативный шаблон (*Varadic Template*) — шаблон с неизвестным числом параметров. Синтаксис вариативного шаблона

```
template<class... Types>
```

где **Types** представляет собой пакет типов. В пакете типов могут присутствовать как параметры-типы, так и **non-type** параметры.

- Пакет типов можно использовать в списке параметров функции:

```
template<class... Types>
void func(Types... parameters)
```

где `parameters` представляет собой пакет параметров, типы которых задаются указанными пакетом типов `Types`. Пакет параметров может быть только в функции-шаблоне, и он должен соответствовать пакету типов.

- Внутри функции можно использовать как пакет типов, так и пакет параметров. Для этого после имени пакета ставится многоточие `...`, которое интерпретируется компилятором как раскрытие пакета в список (через запятую) в указанном месте. Обычным способом определения функций-шаблонов с переменным числом параметров является рекурсивное определение.

Раздел 4. Использование `std::initializer_list`

- Для инициализации массивов (и статических, и динамических) весьма удобно использовать список инициализации:

```
int array[] { 7, 6, 5, 4, 3, 2, 1 };
```

- Но в создаваемых классах список инициализации по умолчанию не работает. Компилятор C++ (начиная с версии C++11) автоматически конвертирует список инициализации в объект типа `std::initializer_list`. Следовательно, для того, чтобы объект класса мог инициализироваться списком, необходимо и достаточно в этом классе создать конструктор, который прини-

мает в качестве параметра объект типа `std::initializer_list`, например,

```
IntArray(const std::initializer_list<int>& list)
```

- Для работы с `std::initializer_list` нужно подключить заголовочный файл `<initializer_list>`.
- Возможность использования списка инициализации таким же образом можно добавить и в шаблон класса.

Раздел 5. Класс `string`

- ASCII представляет собой 7-битную кодировку управляющих символов, символов латинского алфавита, цифр и знаков препинания с кодами от 0 до 127.
- Для национальных алфавитов было создано множество 8-битных кодировок, в которых первые 127 символов совпадают с ASCII, а во второй половине таблицы (от 128 до 255) размещались символы других алфавитов.
- **Unicode** — это таблица кодов символов. Она состоит из 1 114 112 позиций, которые сгруппированы в 17 блоков по 65536 символов. Нулевой блок — основной, он содержит символы всех современных алфавитов.
- На базе Unicode существуют следующие кодировки: UTF-8, UTF-16, UTF-32. Они различаются количеством байтов, отведенных под код одного символа.
- В языке C++ есть следующие (встроенные) типы данных для хранения кода символа:

```
char, char8_t, char16_t, char32_t
```

- Для хранения данных, представленных различными типами символов в C++, соответственно, имеются различные типы строк: `string` с элементами типа `char`, `wstring` с элементами типа `wchar_t`, `u16string` с элементами типа `char16_t`, `u32string` с элементами типа `char32_t`.
- Преимущества класса `string` по сравнению с Си-строками:
 - ▷ объект класса `string` является динамическим массивом, таким образом его легко редактировать;
 - ▷ работа с этим объектом защищена от ошибок адресации;
 - ▷ для работы с объектами `string` в классе существует большой набор полезных и удобных функций;
 - ▷ класс `string` является контейнером, для работы с объектами `string` можно использовать полный набор функций и операций, общий для всех контейнеров.
- Неинициализированный объект `string` получает пустое (" ") значение. Инициализировать строки можно:
 - ▷ константным массивом символов

```
string text{ "Hello" };
```

- ▷ повторением символов (скобки — круглые!)

```
string name( 8, 'a' );
```

- ▷ использованием оператора присвоения

```
string month = "March";
```

- Для вывода строк используется оператор вывода `<<`. Для ввода строк используются оператор ввода `>>` и функция

```
istream getline(istream instream, string str, char delim)
```

- Для объектов класса `string` определены следующие операторы:
 - ▷ присваивания — `=`;
 - ▷ ввода-вывода — `<<`, `>>`;
 - ▷ конкатенации (сложения строк) — `+`, `+=`;
 - ▷ сравнения — `==`, `!=`, `<`, `>`, `<=`, `>=`;
 - ▷ индексирования — `[]`.
- Методы доступа к символам строки

<code>at()</code>	Получение указанного символа с проверкой выхода индекса за границы
<code>front()</code>	Получение первого символа (возвращается ссылка на символ)
<code>back()</code>	Получение последнего символа (возвращается ссылка на символ)
<code>data()</code>	Возвращает константный указатель на лежащий в основе строки C-массив, такой, что <code>data() + i == &operator[](i)</code> для каждого <code>i</code> в <code>[0, size() - 1]</code>

- Методы — итераторы

<code>begin()</code> <code>cbegin()</code>	Возвращает итератор на первый элемент
<code>end()</code> <code>cend()</code>	Возвращает итератор на элемент, следующий за последним
<code>rbegin()</code> <code>rend()</code> <code>crbegin()</code> <code>crend()</code>	Возвращают реверсивные итераторы (для прохода по строке в обратном порядке)

■ Методы поиска в строке:

<code>find()</code>	Находит первую подстроку, равную переданной последовательности символов. Поиск начинается с заданной позиции.
<code>rfind()</code>	Находит последнюю подстроку, равную переданной символьной последовательности. Поиск начинается с заданной позиции.
<code>find_first_of()</code>	Находит первый символ строки, равный одному из символов в переданной последовательности символов. Поиск начинается с заданной позиции.
<code>find_last_of()</code>	Находит последний символ, равный одному из символов в переданной последовательности символов. Поиск начинается с заданной позиции.
<code>find_first_not_of()</code>	Находит первый символ строки, не равный ни одному из символов в переданной последовательности символов. Поиск начинается с заданной позиции.
<code>find_last_not_of()</code>	Находит последний символ, не равный ни одному из символов в переданной последовательности символов. Поиск начинается с заданной позиции.

■ Методы редактирования строки

<code>assign()</code>	Заменяет содержимое строки
<code>clear()</code>	Удаляет все символы строки
<code>insert()</code>	Вставляет символы в строку
<code>erase()</code>	Удаляет указанные символы из строки
<code>push_back()</code>	Добавляет символ к концу строки
<code>pop_back()</code>	Удаляет последний символ строки
<code>append()</code>	Добавляет символы в конец строки
<code>replace()</code>	Заменяет часть строки на указанную подстроку
<code>substr()</code>	Возвращает подстроку
<code>copy()</code>	Копирует подстроку
<code>resize()</code>	Изменяет размер строки
<code>swap()</code>	Обменивает содержимое строки с другой строкой

■ Методы работы со вместимостью

<code>empty()</code>	Проверяет, является ли строка пустой
<code>size()</code> <code>length()</code>	Возвращает количество символов в строке
<code>max_size()</code>	Возвращает максимальное количество элементов, которое может содержать строка
<code>reserve()</code>	Задаёт ёмкость строки, минимум <code>size()</code> . Новая память для хранения выделяется при необходимости
<code>capacity()</code>	Возвращает количество символов, под которые у строки есть выделенное место.
<code>shrink_to_fit()</code>	Запрос для уменьшения ёмкости <code>capacity()</code> до <code>size()</code>

■ Сервисные функции (не члены класса)

<code>getline()</code>	Считывает данные из потока до разделителя или до предельного количества символов.
<code>stoi()</code> <code>stol()</code> <code>stoll()</code>	Преобразует строку в целое (<code>int</code>), длинное целое (<code>long</code>), «сверхдлинное» целое (<code>long long</code>)
<code>stoul()</code> <code>stoull()</code>	Преобразует строку в беззнаковое длинное целое (<code>long</code> , <code>long long</code>)
<code>stof()</code> <code>stod()</code> <code>stold()</code>	Преобразует строку в число с плавающей точкой (<code>float</code> , <code>double</code> , <code>long double</code>)
<code>to_string()</code>	Преобразует целое или число с плавающей точкой в строку

7. Домашнее задание

1. Сжатие текста

В строке текста записаны слова, разделенные пробелами в произвольном количестве. Сжатие текста состоит в том, что между словами оставляется по одному пробелу, а после последнего слова пробелы удаляются (пробелы перед первым словом сохраняются). Если строка содержит только пробелы, то все они сохраняются.

Написать функцию, сжимающую текст.

2. Удаление комментариев

Дан текст программы на C++. Комментарии начинаются с `"//"` и продолжаются до конца текущей строки или начинаются с `"/*"` и заканчиваются `"*/"`. В последнем случае комментарии могут быть в середине строки или располагаться на нескольких строках.

Написать функцию, удаляющую комментарии.

3. Поиск и выделение

Написать функцию, принимающую в качестве параметров две строки и возвращающую копию первого параметра, все вхождения второго параметра в которой взяты в «`()`»

Например, если параметрами были строки

```
"abracadabra" и "ab",
```

то вернуть надо

```
"(ab)aracad(ab)ra".
```

4. Вычисление сумм

В строке текста встречаются выражения, представляющие суммы и разности двух натуральных чисел в виде n_1+n_2 , n_1-n_2 .

Написать функцию, заменяющую выражения их числовыми значениями.

Например, для строки

```
"alpha 5+26 beta 72-35 gamma 32+45 etc"
```

результат должен быть

```
"alpha 31 beta 37 gamma 77 etc".
```

5. Римские числа

Написать функцию, которая принимает строку — запись натурального числа (до 4000) и возвращает строку, содержащую запись этого числа римскими цифрами.

Например, из строки

```
"961"
```

должна получиться строка

```
"CMLXI"
```

6. Полное имя (путь) файла

Написать функцию, которая принимает путь файла и возвращает:

- Путь без имени файла (если пути не было — вывести пустую строку);

- b) Имя последней папки в пути;
- c) Имя файла с расширением, но без пути;
- d) Расширение имени файла;
- e) Имя файла без пути и без расширения.

Например, из строки `"C:\Step\C++lesson_03\Docs\Less03.docx"` должно получиться:

- a) `"C:\Step\C++lesson_03\Docs"`;
- b) `"Docs"`;
- c) `"Less03.docx"`;
- d) `".docx"`;
- e) `"Less03"`.

8. Терминология

- ASCII
- class template
- function template
- `std::initializer_list`
- `std::string`
- STL — Standard Template Library
- string
- UTF-8
- UTF-16
- UTF-32
- Unicode
- template
- varadic template
- vector
- библиотека стандартных шаблонов
- вариативный шаблон
- динамический полиморфизм
- значение типа по умолчанию
- итератор
- обобщенное программирование
- пакет параметров
- пакет типов
- параметр non-type
- параметр-тип
- полиморфизм
- полная специализация шаблона класса
- специализация шаблона
- си-строки
- статический полиморфизм
- универсальное программирование
- частичная специализация шаблона класса
- шаблон
- шаблон класса
- шаблон с переменным числом параметров
- шаблон функции
- явная специализация шаблона класса.

